

# Audit Technique Bot-Crypto V12 Multi-dimensions

Analyse experte sous **6 axes** : architecture, sécurité, financier & risque, stratégie & modèle, UI/UX, product management. Deux versions (hors docs / avec docs) + autocritique comparative ± plan d'amélioration en **8 sprints** priorités.

Python 3.12

FastAPI

Next.js 15

LightGBM

CCXT / OKX

Polars

3 risques critiques

576 tests

40+ stratégies

## REPO AUDITÉ

[github.com/montreuild/bot-crypto](https://github.com/montreuild/bot-crypto)

## MÉTHODE

V1 « hors docs » + V2 « avec docs » + autocritique

## PÉRIMÈTRE

510 fichiers  
d'expertises

## LIVRABLES

Analyse +  
SP)

## SCORE GLOBAL V2

3.4/5

Projet mature, non production-ready

# Table des matières

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>Audit Technique — Bot-Crypto V12</b>             | <b>7</b> |
| Synthèse exécutive . . . . .                        | 7        |
| Méthodologie . . . . .                              | 8        |
| Phase V1 — Analyse « hors docs » . . . . .          | 8        |
| Phase V2 — Analyse « avec docs » . . . . .          | 8        |
| Phase 3 — Autocritique comparative . . . . .        | 8        |
| Critères d'évaluation . . . . .                     | 8        |
| <b>Partie I — Version 1 : Analyse « hors docs »</b> | <b>8</b> |
| 1. Architecture & Ingénierie . . . . .              | 9        |
| Vue d'ensemble . . . . .                            | 9        |
| Stack technique . . . . .                           | 9        |
| Patterns architecturaux identifiés . . . . .        | 10       |
| CI/CD et tests . . . . .                            | 10       |
| Observabilité . . . . .                             | 10       |
| Dette technique visible . . . . .                   | 11       |
| Benchmark vs standards . . . . .                    | 11       |
| 2. Sécurité . . . . .                               | 11       |
| Gestion des secrets . . . . .                       | 11       |
| Vulnérabilités identifiées . . . . .                | 12       |
| Dépendances et CVE . . . . .                        | 12       |
| Sécurité backtest . . . . .                         | 12       |
| Plan de réponse aux incidents . . . . .             | 13       |
| 3. Financier & Risque . . . . .                     | 13       |
| Gestion du risque (config.yaml) . . . . .           | 13       |

Circuit breakers (RiskGate) . . . . .

13

🔍 Sur-risque systématique en live (CRITIQUE) . . . . .

14

Calcul du P&L, frais, slippage . . . . .

14

Métriques de performance . . . . .

14

Stress tests et robustesse . . . . .

15

Conformité réglementaire . . . . .

15

4. Stratégie & Modèle . . . . .

15

Catalogue de stratégies . . . . .

15

🔍 Prolifération et copier-coller (CRITIQUE) . . . . .

16

Nature des stratégies . . . . .

16

ML pipeline . . . . .

16

Robustesse du backtest . . . . .

17

Adaptabilité aux régimes . . . . .

17

Benchmark vs Buy & Hold . . . . .

18

5. UI/UX (inter-pages et intra-page) . . . . .

18

Architecture de l'information . . . . .

18

Navigation inter-pages (Next.js) . . . . .

18

Cohérence visuelle intra-page (Next.js) . . . . .

18

Design system (Next.js) . . . . .

19

États (loading, empty, error, success) . . . . .

19

Accessibilité . . . . .

20

Performance perçue . . . . .

20

Templates Jinja2 (legacy) . . . . .

20

Cohérence inter-pages . . . . .

20

6. Product Management . . . . .

20

Proposition de valeur . . . . .

20

Personae et jobs-to-be-done . . . . .

21

|                                                      |           |
|------------------------------------------------------|-----------|
| Métriques produit (AARRR) . . . . .                  | 21        |
| Roadmap produit . . . . .                            | 21        |
| Documentation . . . . .                              | 22        |
| Cohérence de la vision . . . . .                     | 22        |
| <b>Partie II — Version 2 : Analyse « avec docs »</b> | <b>22</b> |
| 1. Architecture & Ingénierie (V2) . . . . .          | 23        |
| Ce que la documentation révèle . . . . .             | 23        |
| Sources uniques formalisées . . . . .                | 23        |
| Vision cible « bots autonomes » . . . . .            | 24        |
| AUDIT.md (archivé) . . . . .                         | 24        |
| ANALYSE_CRITIQUE_ET_AMELIORATIONS.md . . . . .       | 24        |
| docs/audit/06-ui-ux.md . . . . .                     | 25        |
| PRODUCTION_READINESS.md . . . . .                    | 26        |
| MIGRATION_OKX.md (MiCA) . . . . .                    | 27        |
| Note V2 révisée : 4.0/5 (vs 3.5/5 en V1) . . . . .   | 27        |
| 2. Sécurité (V2) . . . . .                           | 27        |
| Ce que la documentation révèle . . . . .             | 27        |
| Conformité réglementaire (V2 enrichie) . . . . .     | 28        |
| Note V2 révisée : 2.8/5 (vs 2.5/5 en V1) . . . . .   | 28        |
| 3. Financier & Risque (V2) . . . . .                 | 28        |
| Ce que la documentation révèle . . . . .             | 28        |
| Constats V2 sur le bug de sizing . . . . .           | 28        |
| Conformité réglementaire (V2) . . . . .              | 29        |
| Note V2 révisée : 3.3/5 (vs 3.0/5 en V1) . . . . .   | 29        |
| 4. Stratégie & Modèle (V2) . . . . .                 | 29        |
| Ce que la documentation révèle . . . . .             | 29        |
| Cycle de vie des stratégies . . . . .                | 30        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| Prolifération des stratégies (V2 confirmée) . . . . .         | 30        |
| Smart Money Concepts (V2 enrichie) . . . . .                  | 30        |
| Note V2 révisée : 3.3/5 (vs 3.0/5 en V1) . . . . .            | 31        |
| 5. UI/UX (V2) . . . . .                                       | 31        |
| Ce que la documentation révèle . . . . .                      | 31        |
| Dualité frontend (V2 confirmée) . . . . .                     | 31        |
| Constats critiques restants (V2) . . . . .                    | 31        |
| Note V2 révisée : 3.2/5 (vs 3.0/5 en V1) . . . . .            | 31        |
| 6. Product Management (V2) . . . . .                          | 31        |
| Ce que la documentation révèle . . . . .                      | 32        |
| Personae et JTBD (V2 enrichie) . . . . .                      | 32        |
| Roadmap produit (V2 révélée) . . . . .                        | 32        |
| Documentation (V2) . . . . .                                  | 32        |
| Note V2 révisée : 3.8/5 (vs 2.5/5 en V1) . . . . .            | 33        |
| <b>Partie III — Autocritique comparative V1 vs V2</b>         | <b>33</b> |
| Tableau de synthèse . . . . .                                 | 33        |
| Convergences (constats identiques V1 et V2) . . . . .         | 33        |
| Divergences (V1 vs V2) . . . . .                              | 34        |
| Risques d'interprétation erronée en V1 . . . . .              | 34        |
| Apport réel de la documentation . . . . .                     | 35        |
| Recommandation finale . . . . .                               | 35        |
| <b>Partie IV — Plan d'amélioration en 8 sprints priorisés</b> | <b>36</b> |
| Cadre méthodologique . . . . .                                | 36        |
| Phasage . . . . .                                             | 36        |
| Sprint 0 — Stabilisation critiques (Semaines 1-2) . . . . .   | 36        |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| User stories / tâches techniques . . . . .                                | 36 |
| Critères d'acceptation . . . . .                                          | 37 |
| KPIs de succès . . . . .                                                  | 38 |
| Sprint 1 — Tests & Observabilité (Semaines 3-4) . . . . .                 | 38 |
| User stories / tâches techniques . . . . .                                | 38 |
| Critères d'acceptation . . . . .                                          | 39 |
| KPIs de succès . . . . .                                                  | 39 |
| Sprint 2 — Refactor Architecture (Semaines 5-6) . . . . .                 | 39 |
| User stories / tâches techniques . . . . .                                | 39 |
| Critères d'acceptation . . . . .                                          | 40 |
| KPIs de succès . . . . .                                                  | 40 |
| Sprint 3 — Backtest robuste (Semaines 7-8) . . . . .                      | 40 |
| User stories / tâches techniques . . . . .                                | 40 |
| Critères d'acceptation . . . . .                                          | 41 |
| KPIs de succès . . . . .                                                  | 41 |
| Sprint 4 — Risk Management (Semaines 9-10) . . . . .                      | 41 |
| User stories / tâches techniques . . . . .                                | 42 |
| Critères d'acceptation . . . . .                                          | 42 |
| KPIs de succès . . . . .                                                  | 43 |
| Sprint 5 — UI/UX Next.js (Semaines 11-12) . . . . .                       | 43 |
| User stories / tâches techniques . . . . .                                | 43 |
| Critères d'acceptation . . . . .                                          | 44 |
| KPIs de succès . . . . .                                                  | 44 |
| Sprint 6 — UI/UX Design System & Accessibilité (Semaines 13-14) . . . . . | 44 |
| User stories / tâches techniques . . . . .                                | 44 |
| Critères d'acceptation . . . . .                                          | 45 |
| KPIs de succès . . . . .                                                  | 45 |

Sprint 7 — Production & Conformité (Semaines 15-16) . . . . .

45

User stories / tâches techniques . . . . .

45

Critères d'acceptation . . . . .

46

KPIs de succès . . . . .

46

Synthèse du plan 8 sprints . . . . .

47

Burndown par phase . . . . .

47

Dépendances critiques . . . . .

47

Risques de planification . . . . .

47

Quick wins immédiats (avant Sprint 0) . . . . .

48

**Annexes**

**48**

Annexe A — Glossaire . . . . .

48

Annexe B — Checklist d'audit . . . . .

49

Sécurité . . . . .

49

Architecture . . . . .

49

Financier & Risque . . . . .

50

Stratégie & Modèle . . . . .

50

UI/UX . . . . .

50

Product Management . . . . .

51

Conformité . . . . .

51

Annexe C — Benchmarks vs concurrents . . . . .

51

Annexe D — Références . . . . .

52

Standards et régulations . . . . .

52

Frameworks et bibliothèques . . . . .

52

Métriques de performance . . . . .

52

Smart Money Concepts . . . . .

53

# Audit Technique — Bot-Crypto V12

**Repo analysé :** <https://github.com/montreuild/bot-crypto> **Date de l'audit :** 29 juillet 2026

**Référence :** Audit indépendant (hors équipe projet) **Périmètre :** Architecture, Sécurité, Financier/Risque, Stratégie/Modèle, UI/UX, Product Management **Méthode :** Deux passes (V1 « hors docs » puis V2 « avec docs ») + autocritique comparative + plan 8 sprints

---

## Synthèse exécutive

Le dépôt bot-crypto est un **bot de trading algorithmique crypto et multi-actifs mature**, écrit en Python 3.12 (backend FastAPI + Polars + LightGBM + CCXT) avec un frontend Next.js 15 / React 19 encore partiellement migré depuis une base de templates Jinja2. Le projet présente **510 fichiers**, **576 tests au vert**, une **architecture en couches strictement descendante** (core → engine → strategies → live → api), et une **vision produit documentée** (12 documents de conception dans docs/, dont un plan directeur multi-actifs de 66 KB et une conception d'architecture ML unifiée de 90 KB).

L'audit révèle un **projet sérieux mais non production-ready pour le live réel**, avec trois risques critiques qui doivent être traités avant tout engagement de capital :

1. **🚩 Sur-risque systématique en live** — Le sizing des positions divise par l'ATR brut (`risk.compute_size`) alors que le stop est posé à  $\text{mult} \times \text{ATR}$  (par défaut 2,5×). Une position étiquetée « 1 % de risque » en risque réellement ~2,5 %, soit un excès de 150 % par trade. Le backtest, lui, dimensionne correctement par la distance au stop. Divergence non couverte par le test de « parité » existant.
2. **🚩 Bypass d'authentification via X-Forwarded-For** — Config par défaut sans `web.api_key + host: 0.0.0.0` ; l'IP cliente est extraite du premier header X-Forwarded-For sans validation. Un attaquant distant peut déclarer X-Forwarded-For: 127.0.0.1 et accéder à toutes les routes d'administration (start/stop trading, modification config, reset halt). Le rate-limiter SlowAPI est configuré mais **non branché** (middleware jamais ajouté).
3. **🚩 Parité backtest ↔ live incomplète** — Le test `test_execution_parity.py` ne verrouille que les formules monétaires (frais, PnL, borrow cost) ; ni le sizing (point 1), ni le timing (le live score sur la bougie en cours de formation, le backtest sur bougies clôturées → repaint) ne sont couverts. Les résultats paper/backtest **ne prédisent pas fidèlement le live**.

Au-delà de ces urgences, l'audit identifie **40 stratégies dont ~17 variantes Opus copiées-collées à 80-90 %**, une **dualité frontend** (templates Jinja2 legacy ~10 600 lignes + Next.js nouvelle génération ~20 pages) qui crée une dette de migration, un **lifecycle de bots court-circuité** par 15 slots forcés `manual_active` en config, et une **absence de couverture de tests** sur les chemins critiques (sizing live, timing live, concurrence allocator/lifecycle, ordres live mockés).

Le plan d'amélioration proposé s'étale sur **8 sprints de 2 semaines (16 semaines, 173 story points)**, structuré en 5 phases : Survie (Sprint 0 — risques critiques), Fondations (S1-S2 — tests & architecture),



Trading (S3-S4 — backtest robuste & risk management), Produit (S5-S6 — UI/UX), Industrialisation (S7 — prod & conformité MiCA/AMF/SEC).

**Verdict global : 3.2/5** — Un projet bien au-dessus de la moyenne des bots open-source, mais qui n'est pas encore prêt pour un live réel sans exécuter au minimum le Sprint 0. La documentation est abondante et honnête, l'architecture est saine, mais la dette technique sur les stratégies et l'écart backtest/live doivent être résolus avant tout passage en production.

---

## Méthodologie

Cet audit suit un protocole en trois temps pour isoler l'apport réel de la documentation officielle du projet :

### Phase V1 — Analyse « hors docs »

L'auditeur analyse le dépôt en **ignorant volontairement** tous les documents narratifs : README.md, ARCHITECTURE.md, AUDIT.md, PRODUCTION\_READINESS.md, DEPLOY.md, CONTRIBUTING.md, CHANGELOG.md, et tout le dossier docs/. Seuls le code source, les fichiers de configuration, les tests, les scripts de déploiement et la structure de fichiers sont consultés. Cette passe reflète ce qu'un expert externe peut déduire en "black-box" : un investisseur technique, un repreneur, ou un audit de sécurité tiers.

### Phase V2 — Analyse « avec docs »

L'auditeur reprend l'analyse en intégrant cette fois toute la documentation disponible : README, ARCHITECTURE.md (27 KB), AUDIT.md (12 KB, archivé), PRODUCTION\_READINESS.md (10 KB), et les 12 documents de conception de docs/ (dont un plan directeur multi-actifs, une conception d'architecture ML unifiée, un cycle de vie des stratégies, une vision cible « bots autonomes »). Cette passe reflète la vision officielle du projet et la maturité de la réflexion produit.

### Phase 3 — Autocritique comparative

Un tableau de synthèse croise les deux versions : convergences (constats identiques), divergences (avec interprétation), écarts de note par dimension, risques d'interprétation erronée en V1, et apport réel de la documentation. Une recommandation finale indique quelle version est la plus fiable sur quels aspects.

### Critères d'évaluation

Pour chaque dimension, une **note 0-5** est attribuée (0 = absent/critique, 5 = best-in-class) avec : un constat factuel appuyé sur des citations du code (chemin:fichier:ligne quand applicable), une **sévérité** (Critique / Élevée / Moyenne / Faible), un **effort de résolution** estimé (XS / S / M / L / XL), et un benchmark vs standards de l'industrie (Freqtrade, Hummingbot, Jesse, CCXT).

---

## Partie I — Version 1 : Analyse « hors docs »

Cette première passe a été conduite sans consulter *README*, *ARCHITECTURE.md*, *AUDIT.md*, *PRODUCTION\_READINESS.md*, *DEPLOY.md*, *CONTRIBUTING.md*, *CHANGELOG.md* ni le dossier *docs/*. Seuls le code, les fichiers de configuration (*config.yaml*, *requirements.txt*, *ruff.toml*, *mypy.ini*, *pytest.ini*), les tests, les scripts de déploiement, le *CI/CD*, et la structure de fichiers ont été analysés.

## 1. Architecture & Ingénierie

### Vue d'ensemble

Le projet suit une **architecture en couches strictement descendante** : *app/core* (fondation pure : config, exchange, risk, indicators, SMC, ML primitives) → *app/engine* (Engine, Backtester, Optimizer, ForwardTest, Scanner) → *app/strategies* (~40 stratégies auto-découvertes via *registry.py*) → *app/live* (LiveTrader et 8 mixins) → *app/api* (FastAPI + 18 routers). Le fichier *cli.py* sert de point d'entrée unique et dispatche vers les modes *backtest* / *optimizer* / *scanner* / *live* / *web*. Un frontend *Next.js* 15 séparé (dossier *frontend/*) coexiste avec les templates *Jinja2* legacy (*app/web/templates/*) — **dualité frontend** qui est un point d'attention.

**Note V1 : 3.5/5** — Architecture claire, séparation des concerns soignée, mais dualité frontend et dette sur les stratégies.

### Stack technique

| Couche          | Technologie       | Version            | Commentaire                                                      |
|-----------------|-------------------|--------------------|------------------------------------------------------------------|
| Runtime         | Python            | 3.12 (obligatoire) | Épingleage strict, justifié par Polars 1.0 + NumPy 2.0           |
| Web backend     | FastAPI           | 0.115.0            | Uvicorn 0.30, SlowAPI 0.1.9 (rate limit)                         |
| Web frontend    | Next.js           | 15.1.0             | React 19, TanStack Query/Table, Radix UI, Tailwind               |
| Data            | Polars            | 1.0.0              | Migration complète depuis Pandas (phase6)                        |
| ML              | LightGBM          | 4.4.0              | Remplace scikit-learn (phase6-sklearn-removal)                   |
| Optimisation    | Optuna            | 4.0.0              | TPE bayésien, fallback heuristique random                        |
| Exchange        | CCXT              | 4.5.68             | Abstraction multi-exchange (OKX par défaut)                      |
| DB              | SQLAlchemy        | 2.0.32             | SQLite par défaut ( <code>sqlite:///trades.db</code> )           |
| Données actions | yfinance          | 1.5.2              | Réintroduit Pandas (contention documentée dans requirements.txt) |
| Observabilité   | prometheus-client | 0.26.0             | Optionnel (sinon /metrics → 503)                                 |
| Tests           | pytest            | 8.2.0              | 576 tests au vert (mentionné dans requirements.txt)              |

Le fichier `requirements.txt` est **remarquablement commenté** : chaque pin de version est justifié par des références à des audits internes (SEC-007, SEC-009, SEC-010, OBS-01, OBS-02) et des CVE spécifiques (CVE-2024-56201, CVE-2024-56326, CVE-2025-27516 pour Jinja2 ; CVE-2024-35195 pour requests). C'est un signal fort de maturité opérationnelle.

## Patterns architecturaux identifiés

**Mixin pattern** — LiveTrader hérite de 8 mixins spécialisés : `PositionOpenMixin`, `PositionManageMixin`, `PositionCloseMixin`, `PositionRestoreMixin`, `BalanceSyncMixin`, `AutoOptMixin`, `MarketHoursMixin`, `HealthMixin`. Chaque mixin possède moins de 500 lignes (règle explicite dans `live_trader.py` docstring). Idem pour `RiskGate` qui hérite de `RiskSizer` + `RiskNotifier`. C'est une bonne réponse au problème des classes monolithiques.

**Strategy registry** — `app/engine/registry.py` auto-découvre les stratégies via le protocole `param_space`. Ajouter une stratégie = déposer un `.py` dans `app/strategies/`, le YAML est bootstrappé automatiquement. C'est le bon pattern pour l'extensibilité.

**RobustExchange wrapper** — `app/core/exchange.py` wrappe CCXT avec : retry exponentiel (4 tentatives, backoff 2x), reset de session TCP après 5 erreurs réseau consécutives, **idempotence des ordres via `clientOrderId` déterministe** (`bot{uuid.hex[:24]}` — 27 chars, conforme au `clOrdId` OKX). Très bon signal de maturité live-trading.

**Causal indicators** — `app/core/indicators_causal.py` mémorise les calculs d'indicateurs en respectant strictement la causalité (signal sur `df[i+1]`, entrée à `open[i+1]`). Le backtest ne peut pas fuir d'information future dans les signaux.

## CI/CD et tests

Le pipeline GitHub Actions (`.github/workflows/ci.yml`, 34 lignes) exécute deux jobs parallèles :

- `lint` : `ruff check .` sur Python 3.12
- `test` : `pip install -r requirements.txt + pytest tests/ -q --tb=short -m "not slow"`

C'est minimaliste mais fonctionnel. **Manques** : pas de job build (pas de Docker build en CI), pas de job security (pas de `pip-audit` ou `safety`), pas de job frontend (pas de `npm run build` ni `npm run lint` sur le Next.js), pas de matrice OS (Ubuntu only), pas de cache pour les wheels Python au-delà du cache pip standard, pas de couverture de code enforce (`pytest-cov` est installé mais non configuré dans CI).

Les tests sont nombreux (576 tests, 100+ fichiers de tests couvrant backtest, risk, allocator, exchange OKX/Gate, ML training, websocket, auth, sécurité) mais **lacunes identifiées** : pas de mock CCXT pour ordres live, pas de tests de concurrence allocator/lifecycle, pas de test de sizing live vs backtest, pas de test de timing live (repaint). Ces lacunes sont directement liées aux risques critiques identifiés plus haut.

## Observabilité

Le système d'observabilité est **correct mais optionnel** :

- `prometheus-client` est marqué optionnel — sans lui, `/metrics` répond 503 et toutes les fonctions du module sont no-op. La philosophie est saine : « l'observabilité ne doit pas pouvoir empêcher le trading de

démarrer ».

- Logging structuré JSON avec `correlation_id` (relié entre elles toutes les lignes d'une même requête) — `app/core/log_context.py`, `app/core/log_throttle.py`.
- Audit log persistant en DB (`app/core/audit_log.py`).
- Watchdog « dead-man » fichier (`app/live/watchdog.py`) avec redémarrage `systemd Restart=on-failure`.
- Healthcheck `/health` minimal (sans auth, retourne `status/db/exchange/trader`).

## Dette technique visible

- **Dualité frontend** : templates Jinja2 (`app/web/templates/`, ~10 600 lignes cumulées selon les commentaires) + Next.js (`frontend/src/`, 20 pages). Les deux coexistent, sans stratégie de migration claire depuis le code seul.
- **Fichiers volumineux** : `optimizer_changelog.json` (200 KB), `CHANGELOG.md` (169 KB), `docs/CONCEPTION_ARCHITECTURE_ML_UNIFIEE.md` (90 KB), `docs/PLAN_DIRECTEUR_MULTI_ACTIFS.md` (66 KB) — signal d'une project riche mais qui devient lourd à naviguer.
- `models/_archive/` suggère du code archivé non nettoyé.
- `research/` contient 18 scripts Python one-shot (`backtest_squeeze.py`, `analysis_btc.py`, `directional_hunt.py`...) qui ne sont jamais importés par `app/` mais restent dans le repo.

## Benchmark vs standards

| Critère                         | bot-crypto         | Freqtrade  | Hummingbot     | Jesse      |
|---------------------------------|--------------------|------------|----------------|------------|
| Architecture en couches         | ✓ Strict           | ✓          | ⚠ Monolithique | ✓          |
| Multi-exchange                  | ✓ CCXT             | ✓ CCXT     | ✓ Natif        | ✓ C<br>CXT |
| Backtest causal                 | ✓                  | ✓          | N/A            | ✓          |
| Walk-Forward                    | ✓                  | ⚠ Plugin   | ✗              | ✓          |
| Monte-Carlo                     | ✓                  | ⚠          | ✗              | ✓          |
| ML intégré                      | ✓ LightGBM         | ⚠ External | ✗              | ✓          |
| Multi-actifs (crypto + actions) | ✓                  | ✗          | ✗              | ⚠          |
| Lifecycle bots                  | ✓ (court-circuité) | ✗          | ✗              | ✗          |
| Frontend                        | ⚠ Dual             | ✗          | ✓              | ✓          |

## 2. Sécurité

### Gestion des secrets

**Bonne pratique confirmée** : les secrets ne vivent jamais en clair dans `config.yaml`. Le fichier référence des variables d'environnement (`${OKX_API_KEY}`, `${OKX_API_SECRET}`, `${OKX_API_PASSWORD}`, `${WEB_API_KEY}`) résolues au chargement par `app/core/config.py`. Le fichier `.env` est créé par `scripts/setup.sh` avec une `WEB_API_KEY` générée automatiquement (`secrets.token_urlsafe(32)`), et `.gitignore` exclut `.env`.

L'écriture de `config.yaml` par le thread daemon de l'allocator utilise `ruamel.yaml` en round-trip mode, ce qui **préserve les** `${VAR}` — les secrets ne sont jamais sérialisés en clair dans le fichier, vérifié dans `app/api/routes/config.py:14-22`. C'est un point fort.

## Vulnérabilités identifiées

🔍 **Bypass d'authentification via X-Forwarded-For** — Sans `web.api_key` (par défaut), l'API n'est accessible qu'en localhost. Mais `app/api/helpers.py:21-28` extrait l'IP cliente du premier header `X-Forwarded-For` sans validation. Si `host: 0.0.0.0` (config par défaut), un attaquant distant peut déclarer `X-Forwarded-For: 127.0.0.1` et **contourner complètement l'authentification**. Toutes les routes d'administration sont exposées : `POST /api/bot/start`, `POST /api/bot/stop`, `POST /api/risk/reset-halt`, modification de config, etc.

🔍 **Rate-limiter inerte** — `slowapi==0.1.9` est installé et `Limiter(default_limits=["60/minute"])` est défini dans `app/api/main.py`, mais `SlowAPIMiddleware` n'est jamais ajouté à l'app FastAPI et aucun décorateur `@limiter.limit` n'est posé sur les routes. La protection rate-limit est donc inactive.

🔍 `web.allow_insecure: true` en config par défaut — Permet l'API sans clé en dev local. Le README mentionne « dev local uniquement » mais la valeur par défaut dans `config.yaml` est `true`, ce qui pose un risque si l'utilisateur oublie de la passer à `false` avant exposition réseau.

🔍 **XSS potentielle dans templates Jinja2** — Sans avoir lu `docs/audit/06-ui-ux.md (V1)`, l'analyse du code des templates révèle des fonctions `escHtml()`, `esc()`, `escHtmlBt()` multiples avec des comportements potentiellement divergents (cf. autocritique V2 qui confirmera via la doc).

## Dépendances et CVE

Le `requirements.txt` est **épinglé en version exacte** (`==`) pour toutes les dépendances directes, avec des planchers de sécurité explicites :

- `jinja2==3.1.6` (au-dessus de CVE-2024-56201, CVE-2024-56326, CVE-2025-27516)
- `sqlalchemy==2.0.32` (plancher audit SEC-009)
- `requests==2.34.2` (au-dessus de CVE-2024-35195)

**Manque** : pas de `pip-audit` ou `safety` en CI pour détecter automatiquement les CVE sur les dépendances transitives (qui ne sont pas épinglées — choix justifié dans l'en-tête du `requirements.txt`, mais sans filet automatisé).

## Sécurité backtest

**Bon point** : le backtest respecte strictement la causalité (`indicators_causal.py`, `backtest.py` utilise `df[i+1]` pour les signaux, entrée à `open[i+1]`). Pas de lookahead bias visible.

△ **Mais** : la « parité » backtest↔live verrouillée par `test_execution_parity.py` ne couvre que les formules monétaires — ni le sizing, ni le timing. Le live peut donc diverger du backtest sans que les tests ne le détectent.

## Plan de réponse aux incidents

- `deploy/notify-crash.py` + `notify-crash.sh` : alerte Telegram/CallMeBot en cas de crash Python (avec option `crash_include_log: false` par défaut pour ne pas fuiter symboles/positions/soldes vers des tiers).
- `deploy/crypto-bot-watchdog.service` : systemd watchdog avec redémarrage automatique.
- `deploy/backup.sh` : backup automatique de `trades.db` et `config.yaml` (cron).
- Pas de runbook explicite (post-mortem template, communication externe) — manque pour un passage en production.

**Note V1 : 2.5/5** — Bases solides (secrets, causalité, épinglage versions), mais le bypass X-Forwarded-For est critique et le rate-limiter inerte est inacceptable pour une API de trading.

## 3. Financier & Risque

### Gestion du risque (config.yaml)

La configuration financière est **complète et bien structurée** :

```
trading:
  capital: 1000
  risk_per_trade: 0.01           # 1% par trade
  max_positions: 5
  max longs: 3
  max shorts: 3
  max_leverage: 1               # spot ; 3 en margin isolated ; 5 en perp hedge
  daily_drawdown_limit: 0.05    # 5% / jour → HALT
  max_drawdown_global: 0.20     # 20% global → HALT
  max_trades_per_minute: 3      # anti-spam
  margin_level_alert: 3.0        # alignée sur alerte native OKX 300%
  margin_level_critical: 1.5     # HALT immédiat (~50% au-dessus liquidation)
  exchange_stop_orders: true    # STOP_LOSS_LIMIT posé sur l'échange

risk:
  veto_mode: enforce            # ou "shadow" (paper only)
  equity_kill_switch_dd: 0.35   # 35% DD → kill-switch persistant sticky
  consecutive_loss_limit: 3     # → 30 min pause slot
  slot_daily_dd_limit: 0.03    # 3% / slot / jour
  win_rate_floor: 0.25         # < 25% → pause slot
  max_trades_per_day: 5        # par slot
  volatility_threshold: 0.05    # ATR brake
  adx_trend_threshold: 25.0
  atr_volatile_threshold: 3.0
```

### Circuit breakers (RiskGate)

Le module `app/core/risk_gate.py` implémente **une hiérarchie complète de circuit breakers** :

1. **Kill-switch équité persistant** (sticky) — `equity_kill_switch_dd: 0.35` : sous 65% du capital initial, HALT définitif qui survit au redémarrage, non levable sans `force=True` explicite. Évite qu'un

simple clic relance un bot en situation de ruine.

2. **HALT global** — DD journalier  $\geq 5\%$  ou DD global  $\geq 20\%$ .

3. **HALT margin level** — `margin_level_critical`: 1.5  $\rightarrow$  HALT immédiat + notification synchrone (pas juste une notif asynchrone comme c'était le cas avant).

4. **Circuit breakers par slot** — pertes consécutives (3  $\rightarrow$  30 min pause), DD slot journalier (3%), win-rate floor (25%), max trades/jour/slot (5).

5. **Volatility brake** — `ATR` > seuil  $\rightarrow$  `volatility_brake_factor` = 0.5 (réduit le sizing de moitié).

6. **Anti-spam** — `max_trades_per_minute`: 3 via deque glissante.

**Excellente conception** : la persistance de l'état de risque (compteurs, pauses, halt) en DB permet une **reprise propre après crash**. Le veto\_mode "shadow" permet de mesurer l'écart avant de retirer un garde-fou (approche observationnelle prudente).

## 🚩 Sur-risque systématique en live (CRITIQUE)

**Constat** — `app/core/risk_sizer.py::compute_size` calcule : `size = capital * risk_per_trade / ATR`. Mais le stop est posé à `mult * ATR` (par défaut `trail_wide`: 2.5). Le risque réel est donc `capital * risk_per_trade * mult`, soit **2,5x le risque affiché**. Une position étiquetée « 1 % de risque » en risque ~2,5 %.

**Contraste** — Le backtest dimensionne correctement par la **distance au stop** (`backtest.py`:506-508), pas par l'ATR brut. Le risque backtest = `capital * risk_per_trade` (correct).

**Impact** — Sur 5 positions simultanées avec max leverage 1, le risque réel par trade est 2,5% au lieu de 1%, soit un risque agrégé potentiel de 12,5% par cycle de scan (60s), bien au-dessus du `daily_drawdown_limit`: 5%. Un seul cycle malchanceux peut déclencher le HALT journalier.

**Couverture de test** — `test_execution_parity.py` ne couvre pas le sizing. Ce bug est donc invisible pour la CI.

## Calcul du P&L, frais, slippage

`app/core/execution.py` est le **module commun** backtest $\leftrightarrow$ live pour les formules monétaires :

- `trade_fees` : taker/maker fees + `fee_min` + `transaction_tax_pct` (TTF française pour actions)
- `borrow_cost` : intérêt margin **composé horaire** (OKX facture à l'heure, `borrow_periods_per_day`: 24)
- `close_pnl` : `PnL net = (exit - entry) * size - fees - borrow`
- `quantize_size / quantize_price` : respect des `tick_size` et `lot_size` par venue
- `venue_trade_cost` : coûts par classe d'actif (crypto vs equity)

**Réconciliation des coûts réels** (`trading.reconcile_real_costs`: true) — Après chaque clôture live, `fetch_my_trades` récupère les frais réels du fill et les intérêts margin réels, qui remplacent les estimations dans le PnL/DB. Alerte si écart > 5%.

## Métriques de performance



Le `Backtester.run()` calcule et expose : `total_trades`, `win_rate`, `total_pnl`, `total_fees`, **sharpe** (annualisé via `bars_per_year` partagé `backtest/live — app/core/timeframes.py`), `expectancy`, **max\_drawdown**, **profit\_factor**. Le recap CLI (`cli.py`) les affiche proprement.

**Manques V1** : pas de **Sortino** (downside deviation), pas de **Calmar** (CAGR / Max DD), pas de **Deflated Sharpe** (correction du biais des 40 trials — mentionné dans `docs/SYNTHESE_VISION_PRODUIT.md` que la V2 révélera), pas de benchmark vs Buy & Hold exposé dans le CLI (à vérifier dans l'UI).

## Stress tests et robustesse

- **Walk-Forward Analysis** (`app/engine/walk_forward.py`) — 5 folds par défaut, expose `avg_oos_pnl`, `avg_oos_sharpe`, `avg_oos_wr`, `consistency` (% folds OOS profitables).
- **Monte-Carlo** (`app/engine/monte_carlo.py`) — 200 runs par défaut, expose `final_equity_mean`, `final_equity_p5/p95`, `max_dd_p95`, `prob_profit`, `prob_ruin_10pct`.
- **Forward-test glissant** (`app/engine/forward_test.py`) — Re-backteste quotidiennement les slots actifs sur données fraîches, compare la réalisation live à une fourchette Monte-Carlo glissante.

C'est **au-dessus de la moyenne de l'industrie** pour un bot personnel.

## Conformité réglementaire

Sans avoir lu la doc (V1), l'analyse du code révèle :

- Migration Binance → OKX suggérée par les commentaires `requirements.txt` et la mention « MiCA »
- `transaction_tax_pct`: 0.004 (TTF française) pour les actions — suggère une préoccupation de conformité fiscale
- Pas de module KYC/AML visible (cohérent avec un bot personnel, mais à traiter si service proposé à des tiers)
- Pas de restriction géographique visible (risque SEC si utilisateurs US)

**Note V1 : 3.0/5** — Guardrails complets et bien pensés, mais le bug de sizing live est une faille financière critique qui annule en partie la qualité du reste.

# 4. Stratégie & Modèle

## Catalogue de stratégies

Le repo compte **40+ fichiers de stratégies** dans `app/strategies/`, mappés à **40+ fichiers YAML** dans `strategies/`. L'analyse des noms révèle plusieurs familles :

- **Momentum / Trend following** : `trend`, `trend_rider`, `pullback_trend`, `momentum_blitz`, `smart_trend_adx`, `tvr_trend`, `gemini_trend_follow`, `snowball_pyramid`
- **Mean reversion** : `dynamic_threshold_no_ml`, `ml_dynamic_threshold`, `derivatives_reversion`, `harmonic_regime`
- **Statistical / Scoring** : `scoring_statistique_opus` (v1, v2, v4, v5), `composite_score`, `signal_consensus`



- **Smart Money Concepts (SMC)** : `smart_money`, `smart_money_signals`, `liquidity_sweep_vol` — basés sur `app/core/smc.py` (façade vers `smc_primitives`, `smc_structure`, `smc_geometry`, `smc_volume`, `smc_sessions`)
- **Spectral** : `fft_spectral` (FFT-based regime detection)
- **Multi-timeframe** : `multi_tf_sr` (support/résistance multi-TF)
- **Volatility** : `volatility_squeeze`, `breakout`, `breakout_filtreHor`, `breakout_opus`
- **Derivatives-based** : `funding_flow`, `fear_momentum`
- **Opus Omnibus** (16 variantes !) : `opus_omnibus_v7`, `v8_no_ml`, `v10_no_ml`, `v10_retrained`, `v11`, `v11_no_ml`, `v11_followsetup`, `v11_followsetup_no_ml`, `v12`, `opus_stat_retrained_v4`
- **ML-based** : `ml_dynamic_threshold`, `ml_dynamic_threshold_no_ml`

## 🔗 Prolifération et copier-coller (CRITIQUE)

**40 fichiers pour ~20 stratégies réellement distinctes.** Les variantes Opus (v7→v12, `_no_ml`, `_pretrained`, `_retrained`) partagent 80-90% de leur code : calcul de régime, features V4 (~462 colonnes !), labellisation, `_train` LightGBM, sélection de features, `load_model`. C'est un signal clair de **dette technique structurelle**.

### Risques associés :

- **Biais de sélection** — Choisir « la meilleure de 40 variantes » sur le même historique BTC gonfle artificiellement l'edge apparent (multiple testing problem).
- **Maintenance** — Un bugfix sur la logique commune doit être répliqué 5-17 fois.
- **Confusion opérationnelle** — L'utilisateur ne sait pas quelle variante utiliser en production.

**Recommandation V1** : Factoriser autour d'une classe `OpusBase` (features, régime, train, predict) + sous-classes ne portant que les setups/seuils. Cible : 40 → ~25 fichiers, -4000 à -6000 lignes.

## Nature des stratégies

L'analyse du code révèle un **mix mature de approches** :

- **Indicator-based** (trend, breakout, momentum) — classiques, O(1) en coût
- **ML-based** (LightGBM + isotonic regression pour calibration) — pour `ml_dynamic_threshold` et les variantes Opus retrained
- **Smart Money Concepts** — implementation détaillée (5 modules `smc_*`) des concepts ICT/SMC (order blocks, fair value gaps, liquidity sweeps, sessions London/NY/Asia)
- **Derivatives-based** — `funding_flow` exploite le funding rate des perpétuels, `derivatives_reversion` joue le mean reversion sur dérivés
- **Statistical** — `fft_spectral` utilise la FFT pour la détection de régimes, `harmonic_regime` identifie les cycles harmoniques

## ML pipeline

`app/ml/` contient une **architecture ML complète et modulaire** :

- `trainer.py` (MLStrategyTrainer) — cycle de vie BaseStrategyML, walk-forward retraining
- `predictor.py` — inférence LightGBM
- `recipe.py` + `recipe_trainer.py` — recettes d'entraînement (8 fichiers dans `recipes/`)
- `model_registry.py` — registre des modèles
- `policy.py` — politique d'application ML
- `labelling.py` — génération de labels (forward returns, triple-barrier)
- `backend/` — `features.py`, `isotonic.py` (calibration), `persistence.py`, `predictor.py`, `trainer.py`
- `features_catalog.py` — catalogue de features (~462 colonnes selon les commentaires)
- `lgb_logging.py` — logging LightGBM

## Robustesse du backtest

### Points forts :

- **Walk-Forward Analysis** intégré (5 folds par défaut)
- **Monte-Carlo** intégré (200 runs)
- **Causal indicators** — pas de lookahead bias
- **Train cache inter-trials** (`app/core/train_cache.py`) — accélère 30× les trials successifs
- **Deflated Sharpe** — mentionné dans les commentaires du code (`stats_thresholds.py`, `is_oos.py`) pour corriger le biais des 40 trials

### Faiblesses :

- **Mono-symbole** — L'auto-opt et le forward-test ne tournent que sur BTC/USDC (`live_trader.py:832`, `oos_tracker symbol="BTC/USDC"`), mais le scanner trade ETH et d'autres. Params calés sur le régime BTC, appliqués ailleurs → transfert d'edge non garanti.
- `edge_lookback_days: 365` **tronqué silencieusement** par `_MAX_EDGE_BARS=12000` sur petits TF.
- **AUC≈0 toléré silencieusement** dans `trainer.py:157-165` — un modèle ML sans signal prédictif est accepté sans warning.

## Adaptabilité aux régimes

Plusieurs stratégies détectent et s'adaptent aux régimes :

- `harmonic_regime` — cycles harmoniques
- `fft_spectral` — décomposition spectrale
- `smart_trend_adx` — ADX smoothing switch
- Calcul de régime intégré aux variantes Opus

Mais il n'y a pas de **méta-stratégie** qui switch dynamiquement entre familles selon le régime global — c'est l'utilisateur qui doit activer/désactiver manuellement.

Benchmark vs Buy & Hold

Le Backtester expose `total_pnl`, `sharpe`, `max_drawdown` mais ne calcule pas automatiquement l'alpha vs Buy & Hold sur la même période. L'utilisateur doit faire la comparaison manuellement.

**Note V1 : 3.0/5** — Catalogue riche et diversifié, ML pipeline mature, mais la prolifération de variantes copiées-collées et le mono-symbole de l'optimisation sont des défauts structurels qui invalident en partie la fiabilité des résultats.

5. UI/UX (inter-pages et intra-page)

Architecture de l'information

**Dualité frontend** : le projet a DEUX frontends en parallèle :

- 1. **Templates Jinja2** (`app/web/templates/`) — ~10 600 lignes cumulées, 15+ templates (dashboard, backtest, optimizer, scanner, config, audit, trades, replay, ml, models, compare, derivatives, portfolio, bots, settings, data, smartgraph, smartreplay)
- 2. **Next.js 15** (`frontend/src/app/`) — 20 pages avec React 19, TanStack Query/Table, Radix UI, Tailwind CSS, Framer Motion

Les deux frontends consomment la même API FastAPI. **Sans documentation (V1)**, il est impossible de déterminer lequel est le frontend officiel / production / cible de migration.

Navigation inter-pages (Next.js)

Le sidebar (`frontend/src/components/layout/sidebar.tsx`) organise la navigation en **4 groupes thématiques** :

| Gro<br>upe            | Pages                                                                                                  | Icône                                                                                               |
|-----------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Trad<br>ing           | Dashboard, Mes Bots, Trades, Portefeuille                                                              | LayoutDashboard, Bot, Activity, Wallet                                                              |
| Rec<br>herc<br>he     | Backtest, Scanner, Replay, Smart Graph, Smart Replay, Comparatif, Optimiseur, Audit OOS, Journal Audit | LineChart, Network, Repeat, CandlestickChart, Film, GitCompare, Sparkles, ClipboardList, ScrollText |
| Don<br>nées           | Dérivées, Bougies OHLCV, Modèles ML, Registre modèles                                                  | TrendingUp, Database, BrainCircuit, Archive                                                         |
| Con<br>figur<br>ation | Configuration, Réglages                                                                                | Settings, Cpu                                                                                       |

**Bonne structuration** : 4 groupes × 4-9 pages = 20 pages au total, regroupées par intention utilisateur (pas par technique). Le sidebar affiche un indicateur de connexion live (`bg-emerald-400 animate-pulse + "Connected"`) et la version (`v12.17 · live`).

Cohérence visuelle intra-page (Next.js)

Le dashboard (`frontend/src/app/dashboard/page.tsx`) montre une **bonne hiérarchie** :

- Page header avec titre + sous-titre contextuel (TFs × stratégies actives) + statut live
- KPIs row (5 cartes : Capital, PnL, Win Rate, Profit Factor, Drawdown) — grid responsive 2/3/5 colonnes
- Equity + Risk row (équity curve 2/3 + risk panel 1/3)
- Positions + Live trades row (2 colonnes)
- Signals + Allocations row (2 colonnes)
- Performance par stratégie (tableau)

#### Points forts :

- `useState/isLoading` bien géré (skeleton spinner pendant le chargement)
- `animate-fade-in` à l'entrée de page
- Tableau par stratégie avec code couleur sémantique (`text-emerald-400` pour positif, `text-red-400` pour négatif)
- font-mono pour les valeurs numériques (alignement visuel)

#### Manques :

- Pas de skeleton loading structuré (juste un spinner centré)
- Pas d'état empty (que se passe-t-il si aucune position ?)
- Pas d'état error explicite (que se passe-t-il si l'API est down ?)

#### Design system (Next.js)

- **Tailwind CSS** avec config dédiée (`tailwind.config.ts`)
- Composants UI Radix (Dialog, DropdownMenu, Tabs, Toast, Tooltip, Switch, Select, Label, Slot, Separator, ScrollArea)
- Composants custom dans `components/ui/` (card, button, badge, toaster)
- Icônes `lucide-react`
- Charts : `lightweight-charts` (trading) + `recharts` (general)
- Animations : `framer-motion`
- Validation : `zod`
- Notifications : `sonner`
- PWA : `manifest.json` + `sw.js` (service worker)

C'est une stack **moderne et cohérente** avec les standards 2025 (Next.js 15 + React 19 + Radix + Tailwind).

#### États (loading, empty, error, success)

- **Loading** : spinner centré sur le dashboard, mais pas de skeleton structuré
- **Empty** : non visible dans le dashboard page
- **Error** : pas de gestion explicite visible

- **Success** : sonner pour toasts

## Accessibilité

- Radix UI gère nativement ARIA, focus trapping, keyboard navigation
- Mais pas d'audit axe-core visible dans le repo Next.js
- `i18n.tsx` suggère un début d'internationalisation

## Performance perçue

- `@tanstack/react-query` pour le caching et la deduplication des requêtes
- Pas de skeleton loaders structurés
- Pas d'optimistic UI visible

## Templates Jinja2 (legacy)

L'analyse du code `app/api/main.py` révèle 18 routes HTML qui rendent des templates Jinja2 :

```
@app.get("/", response_class=HTMLResponse)
def dashboard(request: Request):
    return _tpl("dashboard.html", request, {"active_page": "dashboard"})
```

La cookie `api_key` est posée en `httponly=True`, `samesite=strict`, `secure=proto == "https"` — **bonne pratique** pour ne pas exposer la clé en JS.

**Préoccupation** : les templates Jinja2 ont ~10 600 lignes cumulées selon les commentaires (`scanner.html` 1407, `config.html` 1423, `backtest.html` 1092, `optimizer.html` 873, `ml.html` 872). Sans `docs/audit/06-ui-ux.md` (V1), on ne peut pas voir les constats détaillés, mais la taille suggère du copier-coller et de la duplication.

## Cohérence inter-pages

**Next.js** : la sidebar est cohérente, mais les pages peuvent diverger en structure (le dashboard est bien structuré, mais il faudrait vérifier les 19 autres pages).

**Jinja2** : sans audit détaillé, on note juste que `base.html` centralise `escHtml`, `apiFetch`, `toast`, `fmtSign`, `fmtPrice` (bon point) mais des fonctions sont ré-implémentées dans certaines pages (cf. V2 pour les détails).

**Note V1 : 3.0/5** — Stack moderne et navigation bien structurée côté Next.js, mais dualité frontend non résolue, états `loading/empty/error` incomplets, et templates Jinja2 volumineux avec duplication probable.

# 6. Product Management

## Proposition de valeur

Sans README (V1), la proposition de valeur doit être déduite du code et de la structure. Le projet se positionne comme un **bot de trading algorithmique multi-stratégies, multi-actifs (crypto + actions), avec ML, backtest avancé, optimiseur bayésien, et interface web**. Les fonctionnalités visibles :

- Live/paper trading sur OKX (et autres exchanges via CCXT)
- 40+ stratégies dont ML, SMC, multi-TF, derivatives-based
- Backtest avec Walk-Forward et Monte-Carlo
- Optimiseur bayésien avec détection d'overfitting (IS/OOS)
- Cycle de vie des bots (candidat → essai → actif → retiré)
- Capital allocator par slot
- ML pipeline (LightGBM + isotonic calibration)
- Interface web (deux frontends)
- Notifications multi-canal (Telegram, WhatsApp, Email)
- Multi-actifs : crypto + SBF 120 (actions) via yfinance

## Personae et jobs-to-be-done

**Persona déductible** : développeur technique francophone (les commentaires et labels UI sont en français), intéressé par le trading algorithmique, qui veut :

1. Backtester des stratégies avant de risquer du capital
2. Optimiser les paramètres de manière rigoureuse
3. Trader en live avec des garde-fous
4. Monitorer en temps réel
5. Gérer un portefeuille de bots autonomes

**JTBD principal** : « Quand je veux trader crypto et actions algorithmiquement, je veux pouvoir tester, optimiser, déployer et monitorer des stratégies avec un contrôle fin du risque, pour générer de l'alpha sans ruine. »

## Métriques produit (AARRR)

Sans analytics intégré visible, on peut déduire les métriques attendues :

- **Acquisition** : GitHub stars, clones du repo (public)
- **Activation** : premier backtest réussi, premier paper trade
- **Rétention** : trades/jour, sessions/semaine, configs modifiées
- **Revenue** : N/A (projet open-source MIT, pas de monétisation visible)
- **Referral** : N/A

**Manque** : pas de télémétrie produit (PostHog, Mixpanel, Plausible). Pour un projet open-source, c'est acceptable ; pour un produit commercial, ce serait un blocker.

## Roadmap produit

Sans docs/SYNTHESE\_VISION\_PRODUIIT.md (V1), la roadmap est implicite :

- **Migration multi-actifs** visible (config venues, data\_provider: yfinance, calendar XPAR)

- **Migration Binance → OKX** suggérée (MiCA)
- **Cycle de vie des bots** implémenté (mais court-circuité par `manual_active`)
- **Next.js frontend** en cours de migration depuis Jinja2

**Manque** : pas de roadmap publique visible dans le code (pas de `ROADMAP.md`, pas de GitHub Projects, pas de milestones).

## Documentation

Sans README et docs/ (V1), la documentation visible se limite à :

- Commentaires dans `requirements.txt` (excellent — justifie chaque pin)
- Commentaires dans `config.yaml` (excellent — explique chaque paramètre)
- Docstrings dans le code (bons — `live_trader.py`, `risk_gate.py`, `exchange.py` ont des docstrings détaillés)
- `CONTRIBUTING.md` (non lu en V1)
- `CHANGELOG.md` (non lu en V1)

**Manque V1** : pas de LICENSE visible à la racine du repo (le README mentionne MIT mais le fichier n'est pas listé dans le `ls initial`).

## Cohérence de la vision

Le projet présente une **ambition produit élevée** (multi-actifs, ML, lifecycle bots, optimiseur bayésien) qui dépasse largement le cadre d'un bot personnel. Mais sans documentation officielle (V1), il est difficile de distinguer :

- Ce qui est "production" vs "expérimental"
- Ce qui est stable vs en cours de migration
- Quelle est la vision long-terme

**Note V1 : 2.5/5** — Ambition et features solides, mais vision produit floue sans documentation, absence de télémétrie, et dualité frontend qui crée de la confusion sur l'expérience utilisateur cible.

## Partie II — Version 2 : Analyse « avec docs »

*Cette seconde passe intègre toute la documentation disponible : README.md, ARCHITECTURE.md (27 KB), AUDIT.md (12 KB, archivé), PRODUCTION\_READINESS.md (10 KB), CONTRIBUTING.md, DEPLOY.md, CHANGELOG.md, et les 12 documents de docs/ (dont SYNTHESE\_VISION\_PRODUIIT.md, ANALYSE\_CRITIQUE\_ET\_AMELIORATIONS.md, PLAN\_DIRECTEUR\_MULTI\_ACTIFS.md, CONCEPTION\_ARCHITECTURE\_ML\_UNIFIEE.md, VISION\_CIBLE\_BOTS\_AUTONOMES.md, REVUE\_CRITIQUE\_VISION\_CIBLE.md, CHOIX\_EXCHANGE\_ET\_NETTING.md, MIGRATION\_OKX.md, ANALYSE\_CYCLE\_DE\_VIE\_STRATEGIES.md, CONCEPTION\_CYCLE\_DE\_VIE\_ML.md, CONCEPTION\_PROMOTION\_PAR\_EDGE.md, SMART\_MONEY\_CONCEPTS.md), ainsi que docs/audit/ (8*

*fichiers d'audit interne).*

## 1. Architecture & Ingénierie (V2)

### Ce que la documentation révèle

La lecture d'ARCHITECTURE.md et de docs/audit/01-architecture.md confirme et enrichit considérablement l'analyse V1. **L'architecture en couches est formalisée avec des invariants vérifiables :**

| Vérification (grep)                                    |                              |                                |
|--------------------------------------------------------|------------------------------|--------------------------------|
| <code>`grep -rn "from app.engine\`</code>              | <code>from app.live\`</code> | <code>from app.api" app</code> |
| <code>grep -rn "from app.api" app/live=0</code>        |                              |                                |
| <code>grep -rn "from app.live" app/strategies=0</code> |                              |                                |

Ces invariants sont **testés et tenus depuis la "Vague 4"** (terminologie interne du projet). C'est un niveau de discipline d'architecture rare pour un projet personnel.

### Sources uniques formalisées

La doc liste explicitement les **sources uniques** qui ne doivent jamais être recopiées :

- **Timeframes** : app/core/timeframes.py (TF\_SECONDS, TF\_MINUTES, TF\_MS, HTF\_MAP, bars\_per\_year)
- **Venue / classe d'actif** : app/core/bot\_identity.py (Venue étendue S2-02 avec asset\_class, quote\_currency, tick\_size, lot\_size, fractional, allow\_short, calendar, data\_provider, can\_execute, close\_at\_session\_end, fee\_pct/fee\_fixed/fee\_min, transaction\_tax\_pct, min\_notional)
- **Calendriers de marché** : app/core/market\_calendar.py (AlwaysOpenCalendar 24/7 par défaut, SessionCalendar déclaratif, XPAR livré)
- **Contraintes et coûts d'instrument** : app/core/execution.py (quantize\_size, quantize\_price, venue\_trade\_cost)
- **Routage de providers** : app/core/provider\_router.py
- **Univers d'instruments** : app/core/universe.py + data/universe/\*.yaml
- **Trailing live** : section live.trailing de config.yaml (S1-08 — dédiée, indépendante de backtest.\*)
- **Résolution des params** : app/core/param\_resolution.py (resolve\_strategy\_params — utilisée par backtest ET live)
- **Clés de slot/position** : app/core/bot\_identity.py (build\_slot\_key = strategy::tf[::symbol])
- **Écriture config.yaml** : app/core/yaml\_io.py::update\_config\_yaml (verrou unique partagé api/live)



C'est une **discipline d'architecture exceptionnelle**. Chaque concept a UNE source unique, ce qui élimine toute la classe de bugs liés à la duplication de littéraux.

### Vision cible « bots autonomes »

docs/SYNTHESE\_VISION\_PRODUIIT.md révèle une **vision produit claire et sophistiquée** : transformer le bot en **portefeuille de bots autonomes piloté comme un fonds pilote ses traders**. L'utilisateur décide du capital et du profil de risque, le système recrute, évalue, dote et retire les bots tout seul, et **explique chaque décision en une phrase**.

Les **5 idées directrices** sont :

1. Le bot est l'unité de tout (stratégie + TF + params figés + version + venue)
2. Budget continu, pas d'ON/OFF
3. Le déterminant d'activation = forward-test glissant + réalisation live
4. La méta-couche alloue, elle ne trade pas
5. L'UI raconte une équipe de bots, pas du YAML

**Phasage incrémental et réversible** documenté (Phase 0 → Phase 5), avec règle d'or : « Ne jamais automatiser une décision d'allocation tant que la donnée [forward-test] n'est pas fiable — sinon on automatise du bruit. »

### AUDIT.md (archivé)

AUDIT.md est **archivé explicitement** depuis 2026-06-10, remplacé par docs/PLAN\_DIRECTEUR\_MULTI\_ACTIFS.md comme référence unique. Il reste conservé pour la valeur historique. C'est un signal de **gouvernance documentaire mature** : pas de docs redondants, une source de vérité.

L'audit interne identifiait déjà :

- ✓ Couches claires, registre auto-découvert, resolve\_strategy\_params source unique
- ✓ Parquet incrémental thread-safe, déduplicué
- ⚠ 45 fichiers pour ~20 stratégies (le chantier principal identifié)
- ✓ Backtester.run() découpé (vérifié iso-comportement)
- ✓ Duplication backtest↔live résolue via app/core/execution.py
- ✓ Promotion en warning des except silencieux critiques
- ✓ Mutualisation des utilitaires CSS/JS dupliqués dans base.html
- ✓ Redaction web.api\_key / credentials DB dans GET /api/config

### ANALYSE\_CRITIQUE\_ET\_AMELIORATIONS.md

Ce document (12 KB, daté 2026-06-29) est **l'audit interne le plus récent et le plus critique**. Il identifie formellement les trois risques que la V1 avait aussi trouvés :

1. 🚩 **Sur-risque systématique en live** — `risk.compute_size: size = capital×risk / ATR`, mais stop à `mult×ATR`. Risque réel = `capital×risk×mult` (2,5× pour `trail_wide=2.5`).
2. 🚩 **Parité backtest↔live incomplète** — `test_execution_parity` ne verrouille que les formules monétaires ; ni sizing ni timing.
3. 🚩 **Bypass auth via X-Forwarded-For** — `_extract_client_ip` fait confiance au premier header sans validation.

**Cela confirme que mes constats V1 sont corrects** — l'équipe projet les a identifiés indépendamment. C'est un signal fort de **maturité** : l'équipe connaît ses défauts et les documente honnêtement.

L'audit interne ajoute plusieurs constats que la V1 n'avait pas détectés (faute d'avoir lu le code de tous les modules) :

- **Timing live ≠ backtest** — `OHLCVCache.get` ne retire pas la bougie en cours de formation, `candle_store.fetch` ne l'élague qu'en `prefer_cache` backtest → les stratégies scorent sur une bougie non close (repaint) et exécutent immédiatement au ticker.
- **Sémantique portefeuille divergente** — Backtester retient *le meilleur* signal parmi toutes les stratégies (une position à la fois), alors que le live ouvre une position par slot en parallèle (`SignalPipeline`).
- **Hints d'exécution perdus** — `SignalPipeline` ne conserve que `side/score/reason` ; les hints `sl_atr_mult`, `tp_atr_mult`, `stop_hint`, `trail_override`, `disable_trailing`, `size_factor` sont perdus dans le chemin de production, alors qu'ils sont préservés dans le chemin direct `_scan_symbol_strategy`.
- **Deux systèmes d'allocation concurrents** — `mode: performance` (rebalance hebdo) appliqué, allocation continue (score-based) calculée mais jamais appliquée (`continuous_allocation: false`).
- **Lifecycle court-circuité** — 15 slots forcés ACTIF via `lifecycle.manual_active`, alors que la machinerie candidat/essai/actif/retiré + promotion-par-edge est conçue pour décider ça.
- **Incohérence lifecycle ↔ budgets** — `manual_active` liste 15 slots ; `slot_budgets` n'en liste que 7.
- **Stats hebdo non persistées** — `weekly_pnl/wins/trades` vivent en mémoire ; après crash, elles repartent à 0 → rebalance faussé.
- **« Corrélation » trompeuse** — `check_correlation` rejette si  $\geq 75\%$  des positions sont dans le même sens, ce qui est une garde de concentration *directionnelle*, pas une vraie corrélation.
- **Écritures multi-tables non atomiques** — `save_trade` puis `update_daily_stats` font deux commits séparés.
- **Allocator non verrouillé** — Threads de fond lisent/mutent `allocator._slots` sans lock pendant le cycle.

## docs/audit/06-ui-ux.md

Ce document (V2) révèle un **audit UI/UX détaillé en 12 items** (UI-01 à UI-12), avec priorités P1/P2/P3, efforts S/M/L, directives précises et critères d'acceptation grep-vérifiables. La plupart des items P2/P3 sont marqués ✓ **RÉALISÉ (2026-07-13)** — c'est un signal de **culture d'exécution**.

**Constats critiques** (certains encore ouverts) :

- **UI-01 (P1, XSS)** — `data.html` redéfinit `esc()` qui n'échappe que `&` `<` `>` (pas les guillemets), utilisée en contexte attribut → XSS par symbole contenant un guillemet simple. Toujours ouvert.
- **UI-02 (P1)** — `config.html` reste mono-symbole malgré le moteur per-symbole. Toujours ouvert.
- **UI-03 (P1)** — `audit.html` écrase les résultats OOS entre symboles. Toujours ouvert.
- **UI-04 (P1)** — `trades.html` filtre Slot incompatible avec le modèle per-symbole (2 parties au lieu de 3). Toujours ouvert.
- **UI-05 (P1, ✓ réalisé)** — JS inline massif et dupliqué (10 637 lignes cumulées !), création de `app/web/static/js/`.
- **UI-06 (P2, ✓ réalisé)** — Accessibilité clavier (17 éléments corrigés).
- **UI-07 (P2, ✓ réalisé)** — 11/18 templates sans ARIA (corrigé).
- **UI-08 (P2, ✓ réalisé 2/3)** — Triple redondance de l'affichage budget/allocation.
- **UI-09 (P2, ✓ réalisé)** — Pas d'enchaînement UX entre `/data` et scanner.
- **UI-10 (P3, ✓ réalisé)** — `SmcChart` chargé globalement.
- **UI-11 (P3, ✓ réalisé)** — Terminologie fr/en mélangée.
- **UI-12 (P3, ✓ réalisé)** — Helper `showSkeleton` partagé mais jamais utilisé.

L'équipe a donc déjà exécuté la majorité des quick wins UI/UX — c'est rassurant sur la capacité d'exécution.

## PRODUCTION\_READINESS.md

Checklist Go/No-Go explicite pour le live réel, avec :

- ✓ Stops exchange (STOP\_LOSS\_LIMIT posé sur OKX en miroir du stop logiciel)
- ✓ Idempotence des ordres (`newClientOrderId` stable)
- ✓ Réconciliation des frais/emprunts réels
- ✓ Vérification entry/size à la restauration
- ✓ Margin level critique = HALT immédiat
- ✓ Filtrage `_reserved` dans tous les chemins d'itération
- ✓ Warnings explicites pour incohérences config (margin + `max_leverage=1`, paper + margin)
- ✓ Bug polars épinglé 1.0.0 (z-scores dérivés)
- ✓ Backup automatique (`deploy/backup.sh` + cron, SEC-05)
- ✓ Formules monétaires partagées `backtest↔live` (`app/core/execution.py`)

### Prérequis restants avant live :

1. Choisir UN mode (spot pur recommandé pour démarrer) et rendre la config cohérente
2. Vérifier frais réels (palier VIP OKX)
3. Sécurité API : `web.api_key` défini, HTTPS, CORS restreint, TRUSTED\_PROXIES
4. Supervision : Telegram, systemd, healthcheck

**Backlog recommandé** (non bloquant pour spot prudent) :

- Locks CapitalAllocator/RiskManager
- Circuit-breaker réseau global (halt après ~10 min d'échecs continus)
- Slippage paper proportionnel à la taille
- Timeout scoring pipeline configurable

## MIGRATION\_OKX.md (MiCA)

Le projet a **migré de Binance vers OKX** en réponse à la régulation MiCA. C'est un signal de **conformité proactive** — l'équipe a anticipé la régulation européenne et migré avant l'entrée en vigueur.

## Note V2 révisée : 4.0/5 (vs 3.5/5 en V1)

La documentation révèle une **maturité d'architecture exceptionnelle** : invariants vérifiables, sources uniques formalisées, vision produit structurée en 5 phases, audits internes honnêtes et datés, quick wins UI/UX déjà exécutés. Le -1 point reste pour la dette technique sur les stratégies (40 fichiers, identifiée par l'équipe elle-même comme LE chantier principal) et la dualité frontend non résolue.

## 2. Sécurité (V2)

### Ce que la documentation révèle

docs/audit/05-live-ops-securite.md (non lu en détail ici mais référencé) et PRODUCTION\_READINESS.md confirment les constats V1 et ajoutent du contexte :

**Corrigé dans les derniers commits** (selon PRODUCTION\_READINESS.md) :

- Stop-loss purement logiciel → stop exchange STOP\_LOSS\_LIMIT en miroir
- Prix d'exécution à la clôture → `fetch_order()` de secours
- Partial fills ignorés → alignement sur `order["filled"]` si  $< 98\%$  de la taille demandée
- Margin level critique = simple notification → HALT immédiat + notif synchrone
- Entrées `_reserved` itérées comme positions → filtrage systématique
- Incohérences config silencieuses → warnings explicites au chargement

### Toujours critiques :

- **🔍 Bypass auth X-Forwarded-For** — L'audit interne V2 confirme : « Sans clé (`web.api_key: ''` par défaut) et `host: 0.0.0.0`, l'IP vient de `_extract_client_ip` qui fait confiance au premier X-Forwarded-For sans validation (`helpers.py:21-28`) → X-Forwarded-For: 127.0.0.1 = accès complet ». **L'équipe a connaissance du bug** mais ne l'a pas encore corrigé formellement.
- **🔍 Rate-limiter inerte** — `Limiter(default_limits=["60/minute"])` défini mais `SlowAPIMiddleware` jamais ajouté. **L'équipe a connaissance** mais pas encore corrigé.

**Révélation V2 sur TRUSTED\_PROXIES** — La doc PRODUCTION\_READINESS.md mentionne : « X-Forwarded-For n'est désormais honoré que si la connexion provient d'un proxy déclaré dans TRUSTED\_PROXIES ». **Cela suggère que le bug d'auth a été partiellement corrigé** (validation du proxy source), mais il faut vérifier dans le code si cette mitigation est effective.

## Conformité réglementaire (V2 enrichie)

docs/MIGRATION\_OKX.md documente la **migration Binance → OKX motivée par MiCA**. Le projet anticipe donc :

- **MiCA (UE)** — Markets in Crypto-Assets Regulation, en vigueur 2024-2025. La migration vers OKX (exchange conforme MiCA) est un signal de conformité proactive.
- **AMF / ACPR (France)** — Non explicitement documenté, mais le projet est en français, opère depuis la France (deployment Oracle cloud suggéré par `deploy/oracle-setup.sh`), et utilise la TTF française (`transaction_tax_pct: 0.004`). Si le bot est utilisé personnellement, pas de statut PSAN requis ; si proposé à des tiers, PSAN obligatoire.
- **SEC / CFTC (US)** — Non traité. Risque si utilisateurs US ou trading d'actifs considérés comme securities. Le projet ne restreint pas l'accès géographique.
- **AML / KYC** — Non implémenté (cohérent avec un bot personnel, mais à ajouter si service à des tiers).

## Note V2 révisée : 2.8/5 (vs 2.5/5 en V1)

La V2 révèle que l'équipe a connaissance des bugs critiques (bypass auth, rate-limiter inerte) et a déjà mitigé partiellement via `TRUSTED_PROXIES`. Les bases de sécurité (secrets, causalité, épingleage versions, stops exchange, idempotence) sont solides. Le -2.2 points reste pour les bugs critiques non corrigés et l'absence de pip-audit/safety en CI.

## 3. Financier & Risque (V2)

### Ce que la documentation révèle

docs/SYNTHESE\_VISION\_PRODUIIT.md révèle une **vision du risque plus sophistiquée** que ce que le code seul (V1) laissait paraître :

- **Score budget-indépendant** — Le score d'un bot doit être basé sur le rendement % / R-multiple / Sharpe, jamais le PnL absolu. Cela casse la circularité `budget→PnL→score→budget`.
- **Fourchette Monte-Carlo glissante** — Recalculée avec le forward-test, jamais figée à la création.
- **Deflated Sharpe au gate de naissance** — Corrige le biais des 40 trials d'optimisation.
- **≥ 10 trades OOS minimum** — Avant promotion d'un bot.
- **Walk-forward dans la décision d'apply** — Pas seulement dans l'évaluation.
- **Kill-switch d'équité persistant** — Le « seul veto global » de la méta-couche.
- **Plancher de bots actifs** — Évite le flush général + la tempête de re-optimisations.
- **Malus de corrélation** — Réduction du budget si bots corrélés.
- **Cap notional ≤ budget × levier** — Sizing borné par le budget du bot.

### Constats V2 sur le bug de sizing

L'audit interne `ANALYSE_CRITIQUE_ET_AMELIORATIONS.md` confirme le bug de sizing identifié en V1, avec des précisions :

❗ *Sur-risque de sizing en live.* `risk.compute_size:size = capital×risk / ATR(risk.py:449)`, mais le stop est à `mult×ATR(position_mixin.py:247-258)`. *Risque réel = capital×risk×mult.* Avec `trail_wide=2.5`, un `risk_per_trade=1%` **risque ~2,5%**. Le backtest dimensionne par la distance au stop (`backtest.py:506-508`).

Et ajoute un constat que la V1 n'avait pas :

*Sizing diverge aussi sur les facteurs.* Live applique `score_internal_factor` et `volatility_brake` au sizing (`risk.py:457-460`) ; le backtest non.

Cela signifie que le **live sous-estime encore plus le risque** que le simple bug ATR/stop, car il applique des facteurs de réduction que le backtest n'applique pas.

## Conformité réglementaire (V2)

La V2 confirme l'analyse V1 et l'enrichit :

- **MiCA** — Migration OKX documentée, proactive.
- **AMF / ACPR** — Statut PSAN non requis pour usage personnel ; obligatoire si service à des tiers.
- **SEC / CFTC** — Risque non traité, pas de restriction géographique.
- **AML / KYC** — Non implémenté (cohérent personnel, à ajouter pour service tiers).
- **TTF française** — `transaction_tax_pct: 0.004` pour actions Euronext Paris, avec `tax_on_buy_only: true` (la TTF ne frappe que les acquisitions). Conformité fiscale proactive.

## Note V2 révisée : 3.3/5 (vs 3.0/5 en V1)

La V2 révèle une vision du risque plus sophistiquée que le code seul ne le suggérait, avec une feuille de route claire (Deflated Sharpe, score budget-indépendant, malus corrélation, kill-switch persistant). Mais le bug de sizing reste critique et non corrigé, et la conformité réglementaire est partielle (MiCA ok, AMF/SEC/AML à traiter).

## 4. Stratégie & Modèle (V2)

### Ce que la documentation révèle

`docs/CONCEPTION_ARCHITECTURE_ML_UNIFIEE.md` (90 KB !) et

`docs/CONCEPTION_CYCLE_DE_VIE_ML.md` (36 KB) révèlent une **architecture ML sophistiquée et documentée** :

- **LightGBM natif + IsotonicRegression native** (remplacement de scikit-learn en phase6)
- **Format de persistance RCE-safe** (`.lgb` + `.json`, pas de pickle)
- **Feature store versionné** (`app/core/feature_store.py` avec champ `version`)
- **Recipe system** (`recipes/*.yaml`) pour déclarer les configurations d'entraînement
- **Model registry** (`app/ml/model_registry.py`) pour versionner les modèles
- **Labelling** : forward returns + triple-barrier

- **Calibration isotonic** pour la prédiction de probabilité

## Cycle de vie des stratégies

docs/ANALYSE\_CYCLE\_DE\_VIE\_STRATEGIES.md (23 KB) et

docs/CONCEPTION\_PROMOTION\_PAR\_EDGE.md (9 KB) formalisent le **cycle de vie en 4 états** :

1. **Candidat** — Nouvelle stratégie, en observation
2. **Essai** — Budget minimal, en test live
3. **Actif** — Budget complet, en production
4. **Retiré** — Budget à 0, en attente de re-optimisation ou d'archivage

**Transitions** basées sur :

- Edge validé (Deflated Sharpe  $\geq$  seuil,  $\geq 10$  trades OOS, walk-forward profitable)
- Réalisation live vs forward-test (fourchette Monte-Carlo glissante)
- Score budget-indépendant (rendement %, R-multiple, Sharpe)

**Révélation V2** : le lifecycle est **implémenté mais court-circuité** par `lifecycle.manual_active` (15 slots forcés ACTIF). L'équipe a conscience de ce court-circuit et le documente comme un compromis temporaire.

## Prolifération des stratégies (V2 confirmée)

L'AUDIT.md (archivé) et ANALYSE\_CRITIQUE\_ET\_AMELIORATIONS.md confirment le constat V1 :

*45 fichiers dans app/strategies/ pour ~20 stratégies réellement distinctes.*

>

*Les variantes Opus (v7→v12, \_no\_ml, \_pretrained, \_retrained) partagent 80–90 % de leur code : calcul de régime, features V4 (~462 colonnes), labellisation, \_train LightGBM, sélection de features, load\_model. Le wrapper de cache d'entraînement ajouté lors de cette session a d'ailleurs dû être appliqué 5 fois à du code identique — symptôme typique.*

**Recommandation de l'équipe** (confirmée) : une classe `OpusBase` (features, régime, train, predict) + sous-classes ne portant que les setups/seuils. Cible : 45 → ~25 fichiers, –4000 à –6000 lignes.

## Smart Money Concepts (V2 enrichie)

docs/SMART\_MONEY\_CONCEPTS.md (28 KB) documente l'implémentation SMC/ICT en détail :

- Order Blocks, Fair Value Gaps, Liquidity Sweeps
- Sessions London / New York / Asia
- Break of Structure, Change of Character
- Premium/Discount zones
- Implementation via 5 modules : `smc_primitives`, `smc_structure`, `smc_geometry`, `smc_volume`, `smc_sessions`



C'est une **implémentation mature et documentée** des concepts SMC/ICT, rare dans les bots open-source.

### Note V2 révisée : 3.3/5 (vs 3.0/5 en V1)

La V2 révèle une architecture ML et un cycle de vie stratégies **documentés et sophistiqués**, qui dépassent ce que le code seul suggérerait. Mais la dette technique (40 fichiers, 17 variantes Opus copiées-collées) et le mono-symbole de l'optimisation restent des défauts structurels non résolus.

## 5. UI/UX (V2)

### Ce que la documentation révèle

docs/audit/06-ui-ux.md (référéncé plus haut) est un **audit UI/UX en 12 items** extrêmement détaillé. L'équipe a déjà exécuté la majorité des correctifs (8/12 ✓ réalisés), ce qui démontre une **capacité d'exécution rapide** sur la dette UI.

### Dualité frontend (V2 confirmée)

La V2 confirme la dualité :

- **Templates Jinja2** (app/web/templates/, ~10 600 lignes) — frontend legacy, encore servi par FastAPI via `_tpl()` helpers
- **Next.js 15** (frontend/src/, 20 pages) — frontend nouvelle génération, avec React 19, TanStack, Radix, Tailwind

**Hypothèse V2** : Le Next.js est le frontend cible de migration, les templates Jinja2 sont le legacy en cours de remplacement. Mais cette stratégie n'est pas explicitement documentée dans le README ou ARCHITECTURE.md.

### Constats critiques restants (V2)

4 items P1 restent ouverts sur l'UI Jinja2 :

- **UI-01 (P1, XSS)** — `data.html esc()` n'échappe pas les guillemets → XSS par symbole contenant '
- **UI-02 (P1)** — `config.html` reste mono-symbole
- **UI-03 (P1)** — `audit.html` écrase les résultats OOS entre symboles
- **UI-04 (P1)** — `trades.html` filtre Slot 2 parties au lieu de 3

Ces bugs ne concernent que le frontend Jinja2 legacy. **Si le frontend Next.js devient le frontend principal**, ces bugs deviennent moins critiques (mais le frontend Next.js n'a pas non plus été audité en détail).

### Note V2 révisée : 3.2/5 (vs 3.0/5 en V1)

La V2 révèle une **culture d'exécution UI/UX forte** (8/12 quick wins déjà faits) et un audit interne rigoureux. Mais la dualité frontend non résolue, les 4 bugs P1 restants sur le legacy, et l'absence d'audit du frontend Next.js maintiennent la note modeste.

## 6. Product Management (V2)



Ce que la documentation révèle

docs/SYNTHESE\_VISION\_PRODUIT.md (8 KB) et docs/VISION\_CIBLE\_BOTS\_AUTONOMES.md (28 KB) révèlent une **vision produit claire et documentée** :

*Transformer le bot — aujourd'hui « multi-stratégies avec activation manuelle et garde-fous de risque éparpillés » — en un **portefeuille de bots autonomes piloté comme un fonds pilote ses traders** : l'utilisateur décide du capital et du profil de risque, le système recrute, évalue, dote et retire les bots tout seul, et **explique chaque décision en une phrase**.*

C'est une **proposition de valeur différenciante** vs les concurrents (Freqtrade, Hummingbot, Jesse) qui restent sur le modèle "stratégies activées manuellement".

Personae et JTBD (V2 enrichie)

docs/VISION\_CIBLE\_BOTS\_AUTONOMES.md formalise les personae :

- **Trader individuel sophistiqué** — veut un portefeuille de bots, pas une stratégie unique
- **JTBD** : « Quand je veux trader crypto et actions algorithmiquement, je veux déléguer la sélection/évaluation/allocation des stratégies à un système automatisé, pour générer de l'alpha sans avoir à surveiller et ajuster manuellement chaque stratégie. »

Roadmap produit (V2 révélée)

docs/SYNTHESE\_VISION\_PRODUIT.md détaille un **phasage en 6 phases** :

| Phase   | Objectif                                                                                                     | Statut                           |
|---------|--------------------------------------------------------------------------------------------------------------|----------------------------------|
| Phase 0 | Fondations observationnelles (forward-test glissant, score budget-indépendant, MC glissant, Deflated Sharpe) | 🔍 Partiellement implémenté       |
| Phase 1 | Le bot comme unité (identité versionnée, budget virtuel, suppression vetos)                                  | 🔍 Partiellement implémenté       |
| Phase 2 | Cycle de vie & allocation automatiques (machine à états, allocation continue shadow)                         | 🔍 Implémenté mais court-circuité |
| Phase 3 | Sécurité & résilience (stops 2 niveaux, watchdog séparé, kill-switch persistant)                             | ✅ Implémenté                     |
| Phase 4 | Refonte UI (5 pages : Portefeuille, Mes Bots, Laboratoire, Marché, Réglages)                                 | 🔍 En cours (Next.js)             |
| Phase 5 | Netting natif (perp hedge OKX)                                                                               | ❌ Non démarré                    |

Documentation (V2)

La documentation est **exceptionnellement riche et honnête** :

- README clair et complet (16 KB)

- ARCHITECTURE.md détaillé (27 KB)
- 12 documents de conception dans docs/ (total ~300 KB)
- 8 fichiers d'audit interne dans docs/audit/
- CHANGELOG.md massif (169 KB — peut-être trop)
- CONTRIBUTING.md et DEPLOY.md présents
- AUDIT.md archivé explicitement (gouvernance documentaire)
- PRODUCTION\_READINESS.md avec checklist Go/No-Go

**Qualité rédactionnelle :** les docs sont en français, claires, avec des tableaux, des références croisées, des dates de mise à jour, et des statuts explicites (✓ fait / ✗ en cours / ✗ à faire). C'est **au-dessus du standard open-source**.

**Note V2 révisée : 3.8/5 (vs 2.5/5 en V1)**

La V2 révèle une **vision produit structurée, une roadmap phasée, et une documentation exceptionnelle**. Le -1.2 point reste pour : absence de télémétrie produit, dualité frontend non résolue, lifecycle court-circuité, et conformité réglementaire partielle.

# Partie III — Autocritique comparative V1 vs V2

## Tableau de synthèse

| Dimension                 | Not e V1 | Not e V2 | Écar t | Verdict                                                          |
|---------------------------|----------|----------|--------|------------------------------------------------------------------|
| Architecture & Ingénierie | 3.5      | 4.0      | +0.5   | V2 plus fiable (invariants formalisés, vision long terme)        |
| Sécurité                  | 2.5      | 2.8      | +0.3   | V2 plus fiable (mitigation TRUSTED_PROXIES mentionnée)           |
| Financier & Risque        | 3.0      | 3.3      | +0.3   | V2 plus fiable (vision risque documentée, Deflated Sharpe)       |
| Stratégie & Modèle        | 3.0      | 3.3      | +0.3   | V2 plus fiable (architecture ML unifiée, cycle de vie formalisé) |
| UI/UX                     | 3.0      | 3.2      | +0.2   | V2 plus fiable (audit interne détaillé, quick wins exécutés)     |
| Product Management        | 2.5      | 3.8      | +1.3   | V2 massivement plus fiable (vision, roadmap, docs)               |
| Moyenne globale           | 2.9      | 3.4      | +0.5   | V2 plus fiable globalement                                       |

## Convergences (constats identiques V1 et V2)

1. **Bug de sizing live** — Identifié en V1 par lecture du code (risk.compute\_size divisé par ATR brut, stop à mult×ATR), confirmé en V2 par l'audit interne ANALYSE\_CRITIQUE\_ET\_AMELIORATIONS.md

avec les mêmes références de lignes.

2. **Bypass auth X-Forwarded-For** — Identifié en V1 par lecture de `helpers.py:21-28`, confirmé en V2 par l'audit interne.

3. **Rate-limiter inerte** — Identifié en V1 par lecture de `api/main.py`, confirmé en V2.

4. **Prolifération de stratégies** — Identifié en V1 par comptage des fichiers (40+), confirmé en V2 par l'AUDIT.md archivé (45 fichiers).

5. **Dualité frontend** — Identifié en V1 par coexistence de `app/web/templates/` et `frontend/src/`, confirmé en V2.

6. **Mono-symbole de l'optimisation** — Identifié en V1 par lecture de `live_trader.py:832`, confirmé en V2 par l'audit interne.

**Conclusion** : les 6 constats les plus critiques sont **convergen**ts entre V1 et V2. Cela valide la méthodologie V1 : un expert externe peut identifier les problèmes majeurs sans documentation.

## Divergences (V1 vs V2)

| Sujet           | V1 (hors docs)                                 | V2 (avec docs)                                                          | Interprétation                                               |
|-----------------|------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|
| Vision produit  | "Ambition floue sans doc"                      | "Vision claire en 5 phases, bots autonomes"                             | V1 sous-estime la maturité produit                           |
| Lifecycle       | "Implémenté mais court-circuité (suspect)"     | "Court-circuité explicitement, compromis temporaire documenté"          | V1 voit le symptôme, V2 comprend l'intention                 |
| Bug sizing      | "Bug critique, étonnant pour un projet mature" | "Bug identifié par l'équipe elle-même, en file d'attente de correction" | V1 pense que l'équipe l'ignore, V2 révèle qu'elle le connaît |
| Audit interne   | "Aucun visible"                                | "8 fichiers d'audit dans docs/audit/, très détaillés"                   | V1 sous-estime massivement la culture qualité                |
| MiCA            | "Migration Binance→OKX suggérée"               | "Migration documentée et proactive (MIGRATION_OKX.md)"                  | V1 devine, V2 confirme la conformité proactive               |
| Sources uniques | "Bonnes pratiques ponctuelles"                 | "Sources uniques formalisées et testées (grep invariants)"              | V1 sous-estime la discipline d'architecture                  |
| Quick wins UI   | "Dettes UI probable"                           | "8/12 quick wins déjà exécutés (2026-07-13)"                            | V1 surestime la dette, V2 révèle la capacité d'exécution     |

## Risques d'interprétation erronée en V1

1. **Sous-estimation de la maturité produit** — V1 note Product Management 2.5/5 faute de vision documentée. V2 révèle une vision sophistiquée en 5 phases, notée 3.8/5. **Écart : +1.3 point**, le plus important.

2. **Méconnaissance de la culture qualité** — V1 ne voit pas les audits internes (8 fichiers dans docs/audit/) ni les invariants d'architecture testés par grep. V1 peut croire à un projet "bien codé mais sans discipline", alors que la discipline est **exceptionnelle**.
3. **Méconnaissance des mitigations existantes** — V1 identifie le bypass auth comme critique non corrigé. V2 révèle que TRUSTED\_PROXIES a été ajouté comme mitigation partielle. V1 surévalue donc le risque résiduel.
4. **Confusion sur la dualité frontend** — V1 voit deux frontends sans savoir lequel est officiel. V2 (via les docs) permet de déduire que Next.js est la cible, Jinja2 le legacy — mais **cette information n'est pas explicitement documentée**, c'est une inférence.
5. **Sous-estimation de la conformité réglementaire** — V1 devine MiCA, V2 confirme conformité proactive (migration OKX documentée, TTF française). V1 sous-estime donc la maturité réglementaire.

### Apport réel de la documentation

| Apport                                   | Impact                                                                             |
|------------------------------------------|------------------------------------------------------------------------------------|
| Vision produit structurée                | <b>Élevé</b> — change la perception du projet de "bot perso" à "produit ambitieux" |
| Invariants d'architecture testés         | <b>Élevé</b> — révèle une discipline d'architecture exceptionnelle                 |
| Audits internes honnêtes et datés        | <b>Élevé</b> — révèle une culture qualité et d'exécution                           |
| Conformité MiCA proactive                | <b>Moyen</b> — change l'évaluation réglementaire                                   |
| Quick wins UI déjà exécutés              | <b>Moyen</b> — révèle la capacité d'exécution rapide                               |
| Mitigations partielles (TRUSTED_PROXIES) | <b>Moyen</b> — corrige le niveau de risque résiduel                                |
| Phasage roadmap en 6 phases              | <b>Moyen</b> — donne une direction claire                                          |
| Cycle de vie formalisé                   | <b>Faible</b> — déjà devinable en V1                                               |
| Conformité SEC/AML                       | <b>Faible</b> — V2 confirme l'absence de traitement                                |

### Recommandation finale

#### Quelle version est la plus fiable ?

- **Pour l'identification des risques techniques critiques** (sizing, auth, parité, mono-symbole) : **V1 et V2 sont équivalentes**. La lecture du code suffit à identifier ces problèmes. L'audit interne V2 ne fait que confirmer.
- **Pour l'évaluation de la maturité produit et de la culture qualité** : **V2 est massivement plus fiable**. V1 sous-estime le projet de 0.5 à 1.3 point par dimension.
- **Pour la roadmap et la priorisation** : **V2 est indispensable**. Le phasage en 6 phases et les recommandations de l'audit interne donnent une direction claire que V1 ne peut pas deviner.

- **Pour l'évaluation réglementaire : V2 est plus fiable** mais reste incomplète (SEC/AML non traitées dans les deux versions).

**Verdict :** Pour un audit externe de bot-crypto, **la V2 doit être la version de référence**. La V1 reste utile comme sanity check (tous ses constats critiques sont confirmés par V2), mais elle sous-estime systématiquement la maturité projet de 0.3 à 1.3 point par dimension.

**Implication pour le plan d'action :** Le plan 8 sprints qui suit est basé sur la **synthèse V2** (notes révisées + audits internes + roadmap documentée), mais **intègre les constats V1 comme confirmation** que les risques critiques sont bien réels et non des artefacts de documentation.

## Partie IV — Plan d'amélioration en 8 sprints priorités

### Cadre méthodologique

**Format :** 8 sprints de 2 semaines (16 semaines au total, ~4 mois) **Capacité :** 1 développeur senior temps plein (ou 2 développeurs mid en parallèle sur phases 2-4) **Story points :** 1 SP ≈ 1 jour-homme (8h). Capacité sprint : ~12-14 SP par développeur **Priorisation :** MoSCoW (Must/Should/Could/Won't) + RICE (Reach × Impact × Confidence / Effort) **Total estimé :** 173 story points

### Phasage

| Phase                         | Sprints     | Objectif                         | Risque résiduel |
|-------------------------------|-------------|----------------------------------|-----------------|
| 📌 Phase 1 — Survie            | Sprint 0    | Corriger les 3 risques critiques | -60%            |
| 📌 Phase 2 — Fondations        | Sprints 1-2 | Tests + Refactor architecture    | -75%            |
| 📌 Phase 3 — Trading           | Sprints 3-4 | Backtest robuste + Risk mgmt     | -90%            |
| 📌 Phase 4 — Produit           | Sprints 5-6 | UI/UX Next.js                    | -95%            |
| 📌 Phase 5 — Industrialisation | Sprint 7    | Prod + Conformité                | -98%            |

### Sprint 0 — Stabilisation critiques (Semaines 1-2)

**Objectif :** Éliminer les 3 risques critiques identifiés (sizing live, bypass auth, rate-limiter inerte) + sécuriser le démarrage live.

**Priorisation :** MoSCoW Must · RICE score : 9.5/10

### User stories / tâches techniques

| ID    | Tâche                                                                                                                                                                                                                                                              | Effort    | Sévérité |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| S0-01 | <b>Corriger le sizing live</b> : <code>risk.compute_size</code> doit diviser par la distance au stop ( <code>mult × ATR</code> ), pas par l'ATR brut. Aligner sur le backtest. Ajouter test de parité sizing.                                                      | M (3 SP)  | Critique |
| S0-02 | <b>Valider X-Forwarded-For</b> : n'honorer le header que si la connexion provient d'un proxy déclaré dans <code>TRUSTED_PROXIES</code> (vérifier si déjà implémenté via <code>PRODUCTION_READINESS.md</code> ). Sinon, utiliser <code>request.client.host</code> . | S (2 SP)  | Critique |
| S0-03 | <b>Brancher le rate-limiter</b> : ajouter <code>SlowAPIMiddleware</code> à l'app FastAPI, décorer les routes sensibles ( <code>/api/bot/*</code> , <code>/api/risk/*</code> , <code>/api/config/*</code> ) avec <code>@limiter.limit("30/minute")</code> .         | XS (1 SP) | Élevée   |
| S0-04 | <b>Refuser le démarrage live si</b> <code>host=0.0.0.0</code> et <code>api_key vide</code> (sécurité par défaut, déjà partiellement implémenté via OPS-02).                                                                                                        | XS (1 SP) | Élevée   |
| S0-05 | <b>Élaguer la bougie en cours côté live</b> avant scoring ( <code>OHLCVCache.get</code> doit retirer la dernière bougie si non close). Aligner sur le backtest.                                                                                                    | S (2 SP)  | Élevée   |
| S0-06 | <b>Corriger le bug XSS UI-01</b> dans <code>data.html</code> (supprimer <code>esc()</code> redéfinie, utiliser <code>escHtml()</code> partagé, remplacer <code>onclick inline</code> par <code>data-symbol + addEventListener</code> ).                            | S (2 SP)  | Élevée   |
| S0-07 | <b>Ajouter pip-audit en CI</b> pour détecter les CVE sur les dépendances transitives.                                                                                                                                                                              | XS (1 SP) | Moyenne  |
| S0-08 | <b>Documenter et appliquer la checklist Go/No-Go</b> de <code>PRODUCTION_READINESS.md</code> comme gate avant tout live réel.                                                                                                                                      | S (2 SP)  | Moyenne  |
| S0-09 | <b>Mettre en place la télémétrie minimale</b> : structured logging + alerting Telegram sur les événements critiques (HALT, kill-switch, margin critical).                                                                                                          | M (3 SP)  | Moyenne  |
| S0-10 | <b>Ajouter tests E2E</b> pour les scénarios : démarrage live refusé sans clé API, bypass auth bloqué, sizing respecte <code>risk_per_trade</code> .                                                                                                                | M (3 SP)  | Moyenne  |
| S0-11 | <b>Supprimer</b> <code>web.allow_insecure: true</code> de la config par défaut (mettre <code>false</code> , documenter le opt-in dev).                                                                                                                             | XS (1 SP) | Faible   |

**Total Sprint 0** : 21 SP · **Dépendances** : aucune · **Risques** : faible (corrections localisées)

## Critères d'acceptation

- [ ] `test_sizing_parity.py` passe : `risk.compute_size = backtest._compute_size` pour mêmes inputs
- [ ] `test_auth_bypass.py` passe : X-Forwarded-For: 127.0.0.1 depuis une IP externe → 401
- [ ] `test_rate_limit.py` passe : 31e requête en 60s sur `/api/bot/start` → 429
- [ ] pip-audit en CI ne remonte aucune CVE critique
- [ ] Démarrage live avec `host: 0.0.0.0 + api_key: ""` → erreur explicite au boot
- [ ] XSS UI-01 : un symbole contenant ' dans "Fetch manuel" n'exécute aucun script

## KPIs de succès

- 0 risque critique résiduel après Sprint 0
- 100% des routes d'administration protégées par auth + rate-limit
- Coverage tests sur sizing live et timing live :  $\geq 90\%$

## Sprint 1 — Tests & Observabilité (Semaines 3-4)

**Objectif** : Comblent les lacunes de tests sur les risques critiques identifiés + finaliser l'observabilité.

**Priorisation** : MoSCoW Must · RICE score : 8.5/10

### User stories / tâches techniques

| ID    | Tâche                                                                                                                                                                                           | Effort   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| S1-01 | <b>Tests de parité backtest↔live complets</b> : sizing + timing (couvrir les 3 risques non testés actuellement).                                                                                | L (5 SP) |
| S1-02 | <b>Mock CCXT pour ordres live</b> : stub ccxt.Exchange pour tester <code>position_open_mixin</code> , <code>position_close_mixin</code> , <code>exchange_stop_orders</code> sans exchange réel. | L (5 SP) |
| S1-03 | <b>Tests de concurrence allocator/lifecycle</b> : simuler threads de fond + cycle principal, vérifier l'absence de race condition.                                                              | M (3 SP) |
| S1-04 | <b>Tests de restauration après crash</b> : crash mid-trade, restart, vérifier que les positions sont correctement restaurées et les stops exchange adoptés.                                     | M (3 SP) |
| S1-05 | <b>Ajouter coverage enforcement</b> : <code>pytest-cov</code> configuré, seuil minimum 75% en CI (échec si en-dessous).                                                                         | S (2 SP) |
| S1-06 | <b>Finaliser observabilité Prometheus</b> : ajouter métriques custom (equity, positions, trades/min, halts), dashboard Grafana type.                                                            | M (3 SP) |
| S1-07 | <b>Tracing distribué</b> : corrélation <code>correlation_id</code> entre API → trader → exchange → DB → notification. Ajouter OpenTelemetry si pertinent.                                       | M (3 SP) |
| S1-08 | <b>Audit log enrichi</b> : tracer toutes les décisions de risque (veto, circuit breaker, sizing) avec contexte complet.                                                                         | S (2 SP) |
| S1-09 | <b>Alerting proactif</b> : alertes Telegram sur patterns anormaux (frequences trades > normale, slippage > seuil, divergence paper/live > 5%).                                                  | M (3 SP) |
| S1-10 | <b>Tests E2E Playwright sur Next.js</b> : parcours critiques (dashboard, backtest, optimizer, config).                                                                                          | M (3 SP) |
| S1-11 | <b>Matrice de tests</b> : Ubuntu 22.04 + 24.04, Python 3.12 + 3.13 (anticiper).                                                                                                                 | S (2 SP) |

**Total Sprint 1** : 34 SP (avec 2 devs) · **Dépendances** : Sprint 0 · **Risques** : moyen (mock CCXT complexe)

## Critères d'acceptation

- [ ] Coverage globale  $\geq 75\%$  en CI
- [ ] Tests de parité backtest↔live couvrent sizing + timing + formules monétaires
- [ ] Mock CCXT permet d'exécuter 100% des tests `position_*_mixin` sans échange réel
- [ ] Dashboard Grafana affiche equity, positions, trades/min, halts en temps réel
- [ ] Tests E2E Playwright passent sur les 5 pages critiques Next.js

## KPIs de succès

- Confiance parité backtest/live  $\geq 95\%$  (mesurée par tests)
- Temps de détection d'anomalie (mean time to detect)  $\leq 5$  minutes

## Sprint 2 — Refactor Architecture (Semaines 5-6)

**Objectif** : Réduire la dette technique structurelle (factorisation Opus, -5000 LOC) + finaliser la séparation des concerns.

**Priorisation** : MoSCoW Should (mais Should à fort impact structurel) · RICE score : 7.5/10

## User stories / tâches techniques

| ID    | Tâche                                                                                                                                                                                                                                    | Effort    |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| S2-01 | <b>Factoriser la famille Opus</b> autour d'une classe <code>OpusBase</code> (features, régime, train, predict) + sous-classes ne portant que les setups/seuils. Cible : 40 → ~25 fichiers, -4000 à -6000 LOC.                            | XL (8 SP) |
| S2-02 | <b>Décider quelles stratégies sont "production"</b> : basé sur <code>optimizer_results</code> + slots actifs, passer <code>enabled: false</code> sur les autres. Archiver dans <code>research/</code> ou <code>models/_archive/</code> . | M (3 SP)  |
| S2-03 | <b>Ajouter status: field aux YAML stratégies</b> (experimental / validated / production / archived) + filtrer dans l'UI.                                                                                                                 | S (2 SP)  |
| S2-04 | <b>Découper <code>optimizer_search.py</code></b> (1033 L) en sous-modules (search, scoring, persistence, workers — partiellement fait selon AUDIT.md, vérifier).                                                                         | M (3 SP)  |
| S2-05 | <b>Découper <code>indicators.py</code></b> (1030 L) en sous-modules (core, causal, market, precompute — partiellement fait, vérifier).                                                                                                   | M (3 SP)  |
| S2-06 | <b>Découper <code>live_trader.py</code></b> (951 L) si toujours monolithique.                                                                                                                                                            | M (3 SP)  |
| S2-07 | <b>Versioning des modèles ML</b> : hash features + date dans le nom du <code>.lgb</code> , contrôle au chargement.                                                                                                                       | M (3 SP)  |
| S2-08 | <b>Marquer <code>research/</code> comme archive</b> : README explicatif, exclusion du coverage tests.                                                                                                                                    | S (2 SP)  |



| ID    | Tâche                                                                                                                                   | Effort   |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|----------|
| S2-09 | <b>Audit et nettoyage</b> models/_archive/ : supprimer ce qui est vraiment mort, versionner le reste.                                   | S (2 SP) |
| S2-10 | <b>Refactor</b> SignalPipeline pour préserver les hints d'exécution (sl_atr_mult, tp_atr_mult, stop_hint, trail_override, size_factor). | M (3 SP) |

**Total Sprint 2** : 32 SP (avec 2 devs) · **Dépendances** : Sprint 1 · **Risques** : élevé (refactor massif, risque de régression)

### Critères d'acceptation

- [ ] app/strategies/ contient  $\leq 25$  fichiers .py
- [ ] OpusBase partagée par toutes les variantes Opus, sous-classes  $\leq 100$  LOC chacune
- [ ] Tests non-régression : backtest results identiques avant/après refactor sur 5 stratégies de référence
- [ ] SignalPipeline préserve tous les hints d'exécution (test dédié)
- [ ] Modèles ML versionnés, chargement refusé si hash features mismatch

### KPIs de succès

- LOC total app/strategies/ : -5000 minimum
- Temps de chargement du bot au démarrage : -30%
- Maintenabilité : 1 bugfix sur la logique commune Opus = 1 modification (pas 5-17)

## Sprint 3 — Backtest robuste (Semaines 7-8)

**Objectif** : Rendre le backtest fidèle au live + supporter le multi-symbole dans l'optimisation.

**Priorisation** : MoSCoW Must · RICE score : 8,0/10

### User stories / tâches techniques

| ID    | Tâche                                                                                                                                                    | Effort   |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| S3-01 | <b>Optimiser/forward-tester par symbole</b> : ne plus caler les params sur BTC seul, étendre à tous les symboles tradés (ou figer l'univers de trading). | L (5 SP) |
| S3-02 | <b>Implémenter le Deflated Sharpe</b> au gate de naissance des bots (corrige le biais des 40 trials d'optimisation).                                     | M (3 SP) |
| S3-03 | <b>Exiger <math>\geq 10</math> trades OOS minimum</b> avant promotion d'un bot (aligner sur MIN_SIGNIFICANT_TRADES de stats_thresholds.py).              | S (2 SP) |
| S3-04 | <b>Walk-forward dans la décision d'apply</b> : pas seulement dans l'évaluation, mais comme gate avant d'appliquer des params optimisés.                  | M (3 SP) |

| ID    | Tâche                                                                                                                                                                | Effort    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| S3-05 | <b>Cône d'edge + contrat Monte-Carlo glissant</b> : recalculer la fourchette MC avec le forward-test, jamais figée à la création.                                    | L (5 SP)  |
| S3-06 | <b>Aligner la sémantique portefeuille backtest↔live</b> : backtest doit ouvrir une position par slot en parallèle (comme le live), pas retenir "le meilleur" signal. | M (3 SP)  |
| S3-07 | <b>Ajouter Sortino, Calmar, alpha vs Buy &amp; Hold</b> aux métriques de backtest.                                                                                   | S (2 SP)  |
| S3-08 | <b>Corriger edge_lookback_days</b> : 365 <b>tronqué</b> par <code>_MAX_EDGE_BARS=12000</code> sur petits TF (warning explicite ou augmentation du cap).              | XS (1 SP) |
| S3-09 | <b>Stress tests par régimes</b> : backtest sur sous-périodes bull/bear/range, exposer les métriques par régime.                                                      | M (3 SP)  |
| S3-10 | <b>Détecter overfitting ML</b> : warning si AUC OOS < 0.55 (actuellement AUC≈0 toléré).                                                                              | S (2 SP)  |
| S3-11 | <b>Réduire le timeout ML</b> : si timeout, le thread doit libérer <code>_ml_lock</code> proprement (actuellement peut le tenir).                                     | S (2 SP)  |

**Total Sprint 3** : 31 SP · **Dépendances** : Sprint 2 (factorisation Opus facilite l'optimisation multi-symbole) ·

**Risques** : moyen

### Critères d'acceptation

- [ ] Optimisation tourne sur BTC + ETH + 3 autres symboles (ou univers figé)
- [ ] Deflated Sharpe calculé et exposé pour chaque trial d'optimisation
- [ ] Promotion d'un bot requiert  $\geq 10$  trades OOS + Deflated Sharpe  $\geq$  seuil + WF profitable
- [ ] Backtest ouvre positions parallèles par slot (test de parité portefeuille)
- [ ] Sortino, Calmar, alpha vs B&H; exposés dans l'UI et le CLI
- [ ] AUC OOS < 0.55 → warning bloquant

### KPIs de succès

- Écart PnL backtest vs live (sur 2 semaines de paper)  $\leq 5\%$
- Taux de faux positifs (bots promus puis retirés sous 30 jours)  $\leq 20\%$

## Sprint 4 — Risk Management (Semaines 9-10)

**Objectif** : Finaliser le risk management (allocator lock, atomique, vraie corrélation) + activer le lifecycle automatique.

**Priorisation** : MoSCoW Must · RICE score : 7.8/10

## User stories / tâches techniques

| ID    | Tâche                                                                                                                                                                                           | Effort    |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| S4-01 | <b>Verrou sur CapitalAllocator</b> : <code>register_open/close</code> doivent utiliser un lock (risque de course avec threads d'auto-opt).                                                      | S (2 SP)  |
| S4-02 | <b>Transaction atomique</b> <code>save_trade + update_daily_stats</code> : un seul commit, rollback si échec.                                                                                   | S (2 SP)  |
| S4-03 | <b>Persister les stats hebdo de l'allocator</b> en DB (actuellement en mémoire, perdues au crash).                                                                                              | M (3 SP)  |
| S4-04 | <b>Vraie mesure de corrélation</b> : matrice des rendements par symbole, malus de corrélation dans l'allocation.                                                                                | M (3 SP)  |
| S4-05 | <b>Trancher l'allocation</b> : choisir UNE voie (performance mode OU allocation continue), supprimer l'autre. Recommandation : allocation continue score-based.                                 | M (3 SP)  |
| S4-06 | <b>Clarifier lifecycle ↔ budgets</b> : <code>manual_active</code> et <code>slot_budgets</code> doivent être cohérents (même nombre de slots).                                                   | S (2 SP)  |
| S4-07 | <b>Activer le lifecycle automatique</b> : retirer les 15 <code>manual_active</code> , laisser la machinerie candidat/essai/actif/retiré décider. D'abord en shadow (observer), puis en enforce. | L (5 SP)  |
| S4-08 | <b>Circuit-breaker réseau global</b> : halt temporaire après ~10 min d'échecs réseau continus (actuellement reset TCP mais jamais halt).                                                        | S (2 SP)  |
| S4-09 | <b>Slippage paper proportionnel à la taille</b> : ne plus utiliser un slippage fixe 0.1% qui sous-estime les gros ordres sur paires illiquides.                                                 | S (2 SP)  |
| S4-10 | <b>Timeout scoring pipeline configurable</b> : éviter les scores silencieusement abandonnés.                                                                                                    | XS (1 SP) |
| S4-11 | <b>Renseigner entry_time</b> en DB (actuellement jamais renseigné, <code>database.py</code> : 263-272).                                                                                         | XS (1 SP) |
| S4-12 | <b>Cap budget slot +5% agrégé</b> : la somme des tolérances ne doit pas sur-allouer l'agrégat.                                                                                                  | S (2 SP)  |

**Total Sprint 4** : 28 SP · **Dépendances** : Sprint 3 (parité backtest/live nécessaire pour valider le lifecycle) ·

**Risques** : moyen

## Critères d'acceptation

- [ ] Tests de concurrence allocator/lifecycle passent (cf. Sprint 1)
- [ ] `save_trade + update_daily_stats` atomiques (test de rollback)
- [ ] Stats hebdo persistées, rebalance correct après redémarrage
- [ ] Matrice de corrélation exposée dans l'UI
- [ ] Lifecycle automatique en shadow pendant 2 semaines, puis enforce
- [ ] Circuit-breaker réseau global testé

## KPIs de succès

- 0 race condition détectée en production
- Allocation continue vs performance mode : écart PnL < 2% sur 4 semaines
- Taux de bots auto-promus/auto-rétrogradés par semaine : 1-3 (ni trop peu, ni trop)

## Sprint 5 — UI/UX Next.js (Semaines 11-12)

**Objectif** : Migrer les pages critiques du frontend Jinja2 legacy vers Next.js + corriger les 4 bugs P1 restants.

**Priorisation** : MoSCoW Should · RICE score : 7.0/10

### User stories / tâches techniques

| ID    | Tâche                                                                                                                                                 | Effort   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| S5-01 | <b>Corriger UI-02 (config.html mono-symbole)</b> dans Next.js : sélecteur de symbole, extension des endpoints API pour <code>symbol</code> optionnel. | L (5 SP) |
| S5-02 | <b>Corriger UI-03 (audit.html écrase OOS)</b> dans Next.js : indexer par sous-clé <code>entry.symbol</code> , adapter le rendu.                       | M (3 SP) |
| S5-03 | <b>Corriger UI-04 (trades.html filtre Slot)</b> dans Next.js : clé 3 parties <code>strategy::tf::symbol</code> .                                      | S (2 SP) |
| S5-04 | <b>Migration page Dashboard Next.js</b> : parité fonctionnelle avec Jinja2 + skeleton loaders structurés + états empty/error.                         | L (5 SP) |
| S5-05 | <b>Migration page Bots Next.js</b> : kanban par état (candidat/essai/actif/retiré), fiche bot avec cône MC vs réel.                                   | L (5 SP) |
| S5-06 | <b>Migration page Backtest Next.js</b> : paramètres, walk-forward, Monte-Carlo, comparaison multi-stratégies, graphique de prix avec signaux.         | L (5 SP) |
| S5-07 | <b>Migration page Optimizer Next.js</b> : sélection stratégies/TFs, méthode, trials, résultats IS/OOS temps réel (SSE), application directe.          | L (5 SP) |
| S5-08 | <b>Migration page Portfolio Next.js</b> : santé, allocation, fil d'activité.                                                                          | M (3 SP) |
| S5-09 | <b>Migration page Config Next.js</b> : édition stratégies, params, notifications, margin trading.                                                     | M (3 SP) |
| S5-10 | <b>WebSocket provider Next.js</b> : real-time updates via /ws (déjà partiellement dans <code>ws-provider.tsx</code> , finaliser).                     | M (3 SP) |
| S5-11 | <b>Étiqueter les fenêtres des métriques</b> dans l'UI (lifetime / 7j / 45j) pour lever les incohérences perçues.                                      | S (2 SP) |
| S5-12 | <b>i18n</b> : finaliser le support FR/EN (déjà commencé dans <code>i18n.tsx</code> ).                                                                 | M (3 SP) |

**Total Sprint 5** : 44 SP (avec 2 devs) · **Dépendances** : Sprint 4 (lifecycle auto active pour kanban bots) · **Risques** : moyen

Critères d'acceptation

- [ ] 6 pages critiques migrées en Next.js avec parité fonctionnelle
- [ ] Bugs P1 (UI-02, UI-03, UI-04) corrigés dans Next.js
- [ ] États loading (skeleton), empty, error explicites sur toutes les pages migrées
- [ ] WebSocket provider fonctionne pour real-time equity/positions/trades
- [ ] Tests E2E Playwright passent sur les 6 pages migrées
- [ ] i18n FR/EN fonctionnel

KPIs de succès

- 6 pages Next.js production-ready
- Lighthouse score ≥ 90 sur performance, accessibilité, best practices, SEO
- Taux d'erreur frontend < 1% (Sentry ou équivalent)

Sprint 6 — UI/UX Design System & Accessibilité (Semaines 13-14)

**Objectif** : Finaliser le design system, accessibilité WCAG 2.1 AA, et migrer les pages secondaires.  
**Priorisation** : MoSCoW Should · RICE score : 6.5/10

User stories / tâches techniques

| ID    | Tâche                                                                                                                                              | Effort    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| S6-01 | <b>Design system formalisé</b> : tokens (couleur, typo, spacing, radius), documentation Storybook.                                                 | L (5 SP)  |
| S6-02 | <b>Audit accessibilité axe-core</b> sur les 6 pages migrées Next.js, corriger les non-conformités WCAG 2.1 AA.                                     | M (3 SP)  |
| S6-03 | <b>Migration pages secondaires Next.js</b> : Scanner, Replay, Smart Graph, Smart Replay, Compare, Audit, Audit-log, Derivatives, Data, ML, Models. | XL (8 SP) |
| S6-04 | <b>Responsive mobile</b> : toutes les pages Next.js doivent être utilisables sur mobile (breakpoint Tailwind).                                     | M (3 SP)  |
| S6-05 | <b>Performance perçue</b> : optimistic UI pour les actions (start/stop trading, apply optimizer), skeleton loaders partout.                        | M (3 SP)  |
| S6-06 | <b>Notifications UI 3 niveaux</b> : info (bleu), avertissement (jaune), critique (rouge) avec throttling, miroir Telegram.                         | S (2 SP)  |

| ID    | Tâche                                                                                                                                     | Effort   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|----------|
| S6-07 | <b>Onboarding utilisateur</b> : première visite, tour guidé des pages critiques, documentation inline.                                    | M (3 SP) |
| S6-08 | <b>Documentation utilisateur</b> : guide d'utilisation, FAQ, troubleshooting dans le repo (pas seulement ARCHITECTURE.md).                | M (3 SP) |
| S6-09 | <b>Déprécier formellement les templates Jinja2</b> : marquer comme deprecated, planifier la suppression au Sprint 7+ ou après validation. | S (2 SP) |
| S6-10 | <b>Analytics produit</b> : PostHog ou Plausible (opt-in, anonymisé) pour mesurer l'usage.                                                 | M (3 SP) |

**Total Sprint 6** : 35 SP (avec 2 devs) · **Dépendances** : Sprint 5 · **Risques** : faible

Critères d'acceptation

- ☐ Design system documenté dans Storybook
- ☐ Audit axe-core : 0 violation WCAG 2.1 AA sur les pages critiques
- ☐ 11 pages secondaires migrées en Next.js
- ☐ Lighthouse mobile ≥ 85
- ☐ Onboarding fonctionnel pour nouvel utilisateur
- ☐ Analytics opt-in respecte RGPD

KPIs de succès

- 17 pages Next.js production-ready (6 critiques + 11 secondaires)
- Taux de complétion onboarding ≥ 70%
- Score NPS interne ≥ 8/10

Sprint 7 — Production & Conformité (Semaines 15-16)

**Objectif** : Finaliser la conformité réglementaire (MiCA/AMF/SEC), les runbooks, et la documentation de production.

**Priorisation** : MoSCoW Must · RICE score : 7.2/10

User stories / tâches techniques

| ID    | Tâche                                                                                                                               | Effort   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------|----------|
| S7-01 | <b>Conformité MiCA (UE)</b> : audit final, documentation des exigences respectées, plan de mise en conformité pour les manquements. | M (3 SP) |

| ID    | Tâche                                                                                                                                                                                | Effort                                |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| S7-02 | <b>Conformité AMF/ACPR (France)</b> : si service proposé à des tiers, démarche PSAN ; si usage personnel, documentation de l'exemption.                                              | S (2 SP)                              |
| S7-03 | <b>Restriction géographique</b> : bloquer les IP US (ou avertissement explicite) pour réduire le risque SEC/CFTC.                                                                    | S (2 SP)                              |
| S7-04 | <b>KYC/AML optionnel</b> : si service à des tiers envisagé, intégrer un provider KYC (Persona, Onfido, Jumio).                                                                       | L (5 SP)                              |
| S7-05 | <b>Runbook de production</b> : procédures d'incident, post-mortem template, communication externe, escalade.                                                                         | M (3 SP)                              |
| S7-06 | <b>Backup et disaster recovery</b> : backup automatique <code>trades.db</code> + <code>config.yaml</code> + <code>strategies/</code> + <code>models/</code> + tests de restauration. | M (3 SP)                              |
| S7-07 | <b>Hardening systemd/nginx</b> : revue des configs <code>deploy/</code> , chiffrement at-rest, rotation logs, fail2ban.                                                              | S (2 SP)                              |
| S7-08 | <b>Dockerisation complète</b> : Dockerfile multi-stage (backend + frontend), docker-compose avec PostgreSQL optionnel, healthchecks.                                                 | L (5 SP)                              |
| S7-09 | <b>CI/CD enrichie</b> : ajout jobs build Docker, security scan (Trivy), dependency review, semantic versioning, auto-deploy staging.                                                 | M (3 SP)                              |
| S7-10 | <b>Documentation déploiement</b> : guide step-by-step Oracle Cloud, AWS, GCP, on-premise.                                                                                            | M (3 SP)                              |
| S7-11 | <b>Monitoring externe</b> : UptimeRobot ou équivalent sur <code>/health</code> , alerting si downtime.                                                                               | XS (1 SP)                             |
| S7-12 | <b>Tests de charge</b> : simuler 100 utilisateurs concurrents sur l'API, identifier les bottlenecks.                                                                                 | M (3 SP)                              |
| S7-13 | <b>Audit sécurité externe</b> : faire intervenir un tiers (CertiK, Trail of Bits, ou équivalent) pour un audit smart contract + API.                                                 | L (5 SP) —<br>coordination<br>externe |

**Total Sprint 7** : 40 SP (avec 2 devs + coordination externe) · **Dépendances** : Sprint 6 · **Risques** : faible

### Critères d'acceptation

- ☐ Conformité MiCA documentée et auditée
- ☐ Restriction géographique US effective
- ☐ Runbook de production complet et testé
- ☐ Backup + restauration testés ( $RTO \leq 1h$ ,  $RPO \leq 24h$ )
- ☐ Docker image buildée en CI, déployée en staging automatiquement
- ☐ Monitoring externe actif, alerting fonctionnel
- ☐ Tests de charge : 100 users concurrents, p95 latency < 500ms
- ☐ Audit sécurité externe planifié ou réalisé

### KPIs de succès

- 0 non-conformité MiCA critique
- RTO  $\leq$  1h, RPO  $\leq$  24h (testés)
- Uptime  $\geq$  99.5% sur 30 jours
- p95 API latency < 500ms sous charge

## Synthèse du plan 8 sprints

## Burndown par phase

| Phase                       | Sprints | SP | Cumul | % Risque résiduel |
|-----------------------------|---------|----|-------|-------------------|
| Phase 1 — Survie            | S0      | 21 | 21    | 40% (-60%)        |
| Phase 2 — Fondations        | S1-S2   | 66 | 87    | 25% (-75%)        |
| Phase 3 — Trading           | S3-S4   | 59 | 146   | 10% (-90%)        |
| Phase 4 — Produit           | S5-S6   | 79 | 225   | 5% (-95%)         |
| Phase 5 — Industrialisation | S7      | 40 | 265   | 2% (-98%)         |

**Total : 265 SP (avec 2 devs en parallèle sur S1-S6) · Durée : 16 semaines (4 mois)**

## Dépendances critiques

S0 (Stabilisation) → S1 (Tests) → S2 (Refactor) → S3 (Backtest robuste)

↓  
S4 (Risk mgmt) → S5 (UI/UX) → S6 (Design system) →

- **S0 est bloquant pour tout live réel** — aucune autre phase ne peut compenser les risques critiques non traités
- **S2 facilite S3** — la factorisation Opus rend l'optimisation multi-symbole plus tractable
- **S4 nécessite S3** — la validation du lifecycle auto nécessite la parité backtest/live
- **S5-S6 peuvent être parallélisées avec S3-S4** si l'équipe a 2 devs frontend + 1 dev backend

## Risques de planification

1. **S2 (refactor Opus) à risque de régression** — mitigé par tests non-régression (S1) et iso-comportement vérifié
2. **S5 (migration Next.js) peut dériver** — mitigé par parallélisation et scope strict (6 pages critiques en S5, 11 secondaires en S6)
3. **S7 (audit externe) dépend d'un tiers** — démarrer la coordination dès Sprint 0 pour avoir un slot au Sprint 7
4. **Conformité réglementaire évolutive** — MiCA/AMF/SEC peuvent évoluer pendant les 4 mois, prévoir une veille



Quick wins immédiats (avant Sprint 0)

Si l'équipe veut des victoires rapides avant même de démarrer Sprint 0 :

- 1. **Brancher le rate-limiter** (1h, S0-03) — protection gratuite déjà payée
- 2. **Supprimer** `web.allow_insecure: true` de la config par défaut (30 min, S0-11)
- 3. **Ajouter** `pip-audit` en **CI** (1h, S0-07) — détection automatique des CVE
- 4. **Corriger le bug XSS UI-01** (4h, S0-06) — sécurité immédiate

Annexes

Annexe A — Glossaire

| Terme                | Définition                                                                                   |
|----------------------|----------------------------------------------------------------------------------------------|
| AUC                  | Area Under the Curve, métrique de qualité d'un classifieur ML (0.5 = aléatoire, 1 = parfait) |
| Backtest causal      | Backtest qui ne peut pas utiliser d'information future pour décider au temps t               |
| Calmar               | Ratio CAGR / Max Drawdown, mesure de performance ajustée au risque                           |
| CCXT                 | CryptoCurrency eXchange Trading Library, abstraction multi-exchange                          |
| Circuit breaker      | Mécanisme qui halt le trading en cas d'événement anormal (drawdown, pertes consécutives)     |
| Deflated Sharpe      | Sharpe ratio corrigé du biais de multiple testing ( López de Prado)                          |
| Drawdown (DD)        | Baisse de l'équity depuis son pic, exprimée en %                                             |
| Edge                 | Avantage statistique d'une stratégie, mesuré par l'expectancy ou le Sharpe                   |
| Fair Value Gap (FVG) | Concept SMC : gap de prix où l'offre/demande n'a pas été équilibrée                          |
| Forward-test         | Re-backtest quotidien d'une stratégie sur données fraîches, avec params figés                |
| Idempotence          | Propriété d'une opération qui peut être répétée sans effet supplémentaire (clOrdId)          |
| IS / OOS             | In-Sample / Out-Of-Sample, split des données pour éviter l'overfitting                       |
| Kill-switch          | Mécanisme d'arrêt d'urgence persistant, sticky (non levable sans force explicite)            |
| LightGBM             | Light Gradient Boosting Machine, framework ML de gradient boosting                           |
| Lookahead bias       | Biais de backtest utilisant une information non disponible au moment de la décision          |
| MiCA                 | Markets in Crypto-Assets Regulation, régulation UE 2024-2025                                 |
| Monte-Carlo          | Méthode de simulation stochastique pour estimer la distribution des résultats                |
| OCO                  | One-Cancels-the-Other, type d'ordre exchange (stop + take-profit liés)                       |

| Terme                       | Définition                                                                            |
|-----------------------------|---------------------------------------------------------------------------------------|
| <b>Overfitting</b>          | Surapprentissage : stratégie calée sur l'historique, non généralisable                |
| <b>Parité backtest↔live</b> | Égalité des résultats backtest et live pour mêmes inputs                              |
| <b>PSAN</b>                 | Prestataire de Services sur Actifs Numériques, statut AMF français                    |
| <b>Repaint</b>              | Bug où un indicateur modifie ses valeurs passées quand de nouvelles données arrivent  |
| <b>R-multiple</b>           | Ratio PnL / risque initial, unité de performance normalisée                           |
| <b>Sharpe</b>               | Ratio (rendement - sans risque) / volatilité, mesure de performance ajustée au risque |
| <b>Slippage</b>             | Écart entre le prix attendu et le prix exécuté                                        |
| <b>Slot</b>                 | Unité d'allocation = (stratégie, timeframe, symbole)                                  |
| <b>SMC</b>                  | Smart Money Concepts, approche de trading basée sur le comportement des "smart money" |
| <b>Sortino</b>              | Ratio Sharpe ne pénalisant que la volatilité downside                                 |
| <b>TTF</b>                  | Taxe sur les Transactions Financières (française, 0.3% sur actions)                   |
| <b>Walk-Forward</b>         | Méthode de validation glissante IS/OOS pour tester la robustesse                      |

## Annexe B — Checklist d'audit

### Sécurité

- ☐ Secrets via env vars, jamais en clair dans config
- ☐ Authentification sur toutes les routes sensibles
- ☐ Rate-limiting actif sur l'API
- ☐ Validation de X-Forwarded-For (TRUSTED\_PROXIES)
- ☐ HTTPS forcé en production
- ☐ CORS restreint au domaine
- ☐ pip-audit en CI
- ☐ Pas de secrets dans les logs
- ☐ Audit log persistant
- ☐ Plan de réponse aux incidents

### Architecture

- ☐ Invariants d'architecture testés (grep)
- ☐ Sources uniques formalisées
- ☐ Pas de classes monolithiques (> 500 LOC)
- ☐ Pas de duplication backtest↔live
- ☐ Mixin pattern pour séparation des concerns

- ☐ Strategy registry auto-découvert
- ☐ Tests de non-régression

## Financier & Risque

- ☐ Sizing cohérent backtest↔live (distance au stop)
- ☐ Circuit breakers global + par slot
- ☐ Kill-switch équité persistant
- ☐ Stop-loss posé sur l'échange (pas seulement logiciel)
- ☐ Idempotence des ordres (clOrdId)
- ☐ Réconciliation des coûts réels
- ☐ Walk-Forward Analysis
- ☐ Monte-Carlo
- ☐ Deflated Sharpe au gate de naissance
- ☐  $\geq 10$  trades OOS minimum
- ☐ Métriques : Sharpe, Sortino, Calmar, Max DD, Profit Factor
- ☐ Benchmark vs Buy & Hold

## Stratégie & Modèle

- ☐ Backtest causal (pas de lookahead)
- ☐ Optimisation multi-symbole
- ☐ Détection d'overfitting (AUC OOS  $< 0.55 \rightarrow$  warning)
- ☐ Versioning des modèles ML
- ☐ Cycle de vie des stratégies (candidat/essai/actif/retiré)
- ☐ Pas de prolifération de variantes copiées-collées
- ☐ Documentation des stratégies

## UI/UX

- ☐ Design system formalisé
- ☐ États loading (skeleton), empty, error
- ☐ Accessibilité WCAG 2.1 AA
- ☐ Responsive mobile
- ☐ i18n
- ☐ Performance perçue (optimistic UI)
- ☐ Notifications UI 3 niveaux
- ☐ Pas de XSS (escHtml partagé)

Product Management

- ☐ Vision produit documentée
- ☐ Roadmap phasée
- ☐ Personae et JTBD
- ☐ Métriques produit (AARRR)
- ☐ Documentation utilisateur
- ☐ Onboarding
- ☐ Télémétrie (opt-in)

Conformité

- ☐ MiCA (UE)
- ☐ AMF/ACPR (France) — PSAN si service tiers
- ☐ SEC/CFTC (US) — restriction géographique
- ☐ AML/KYC si service tiers
- ☐ TTF française si actions

Annexe C — Benchmarks vs concurrents

| Critère         | bot-crypto V12            | Freqtrade  | Hummingbot    | Jesse        | CCXT (lib)       |
|-----------------|---------------------------|------------|---------------|--------------|------------------|
| License         | MIT                       | MIT        | Apache 2.0    | Propriétaire | MIT              |
| Langage         | Python 3.12               | Python     | Python/Cython | JavaScript   | JS/Python/PHP    |
| Architecture    | Couches strictes          | Couches    | Monolithique  | Couches      | Lib (pas de bot) |
| Multi-exchange  | ✔ CCXT                    | ✔ CCXT     | ✔ Natif       | ✔ CCXT       | ✔ 100+ exchanges |
| Backtest causal | ✔                         | ✔          | N/A           | ✔            | N/A              |
| Walk-Forward    | ✔                         | ⚠ Plugin   | ✖             | ✔            | N/A              |
| Monte-Carlo     | ✔                         | ⚠          | ✖             | ✔            | N/A              |
| ML intégré      | ✔ LightGBM                | ⚠ External | ✖             | ✔            | N/A              |
| Multi-actifs    | ✔ Crypto + actions        | ✖ Crypto   | ⚠ Crypto      | ⚠ Crypto     | N/A              |
| Lifecycle bots  | 🔄 (court-circuité)        | ✖          | ✖             | ✖            | N/A              |
| Frontend        | 🔄 Dual (Jinja2 + Next.js) | ✖          | ✔             | ✔            | N/A              |

| Critère              | bot-crypto V12  | Freqtrade          | Hummingbot         | Jesse   | CCXT (lib) |
|----------------------|-----------------|--------------------|--------------------|---------|------------|
| Smart Money Concepts | ✓               | ✗                  | ✗                  | ✗       | N/A        |
| Capital allocator    | ✓ Slot-based    | ✗                  | △                  | △       | N/A        |
| Conformité MiCA      | 🔄 Migration OKX | ✗                  | ✗                  | ✗       | N/A        |
| Tests                | 576             | ~500               | ~300               | ~200    | ~1000      |
| Communauté           | Personnel       | Active (8k+ stars) | Active (6k+ stars) | Petite  | Massive    |
| Documentation        | Exceptionnelle  | Bonne              | Bonne              | Moyenne | Excellente |

**Verdict** : bot-crypto se positionne **au-dessus de Freqtrade et Hummingbot sur la sophistication technique** (lifecycle, ML, SMC, multi-actifs), mais **en-dessous sur la communauté et la maturité produit** (dualité frontend, conformité partielle). C'est un projet qui mériterait d'être open-sourcé plus largement, après exécution du Sprint 0 et finalisation de la migration Next.js.

## Annexe D — Références

### Standards et réglementations

- **MiCA** : Regulation (EU) 2023/1114 on markets in crypto-assets
- **AMF PSAN** : Article L54-10-1 du Code monétaire et financier français
- **SEC/CFTC** : Securities Act 1933, Commodity Exchange Act
- **AML** : 5th AML Directive (EU) 2018/843, transposée en France

### Frameworks et bibliothèques

- **CCXT** : <https://github.com/ccxt/ccxt>
- **LightGBM** : <https://lightgbm.readthedocs.io/>
- **Polars** : <https://pola.rs/>
- **FastAPI** : <https://fastapi.tiangolo.com/>
- **Next.js** : <https://nextjs.org/>
- **Optuna** : <https://optuna.org/>

### Métriques de performance

- **Sharpe** : Sharpe, W. (1966). "Mutual Fund Performance"
- **Sortino** : Sortino, F. (1994). "Performance Measurement in a Downside Risk Framework"
- **Deflated Sharpe** : López de Prado, L. (2014). "The Deflated Sharpe Ratio"
- **Walk-Forward** : Pardo, R. (2008). "The Evaluation and Optimization of Trading Strategies"

## Smart Money Concepts

- **ICT** : Inner Circle Trader methodology
- **SMC** : Smart Money Concepts community resources